Part I : Tutorial Getting Acquainted with the LabWindows/CVI Development Environment

1. Introduction to LabWindows/CVI

▶Installing LabWindows/CVI

LabWindows/CVI 개발 환경을 설치한 후 다음의 서브디렉토리를 확인한다.

Directory Name	Contents
c:\cvi\bin	LabWindows/CVI Library files
c:\cvi\extlib	Files for using the LabWindows/CVI libraries with external compilers (Windows 95/NT only)
c:\cvi\fonts	Font files required for graphics operations
c:\cvi\include	Include files associated with libraries
c:\cvi\instr	Instrument modules
c:\cvi\samples	Source code to sample programs
c:\cvi\sdk	SDK library files (Windows 95/NT only)
c:\cvi\toolslib	Additional development tools and libraries
c:\cvi\tutorial	Programs you use in the tutorial sessions throughout this manual.
c:\cvi\vxd vxd	sample code templates (Windows95/NT only)

▶How to proceed

<LabWindows/CVI에 능숙해지는 방법>

1. LabWindows/CVI와 함께 배포된 Release Note와 readme.cvi를 전체적으로 읽는다.
2. 이 장의 나머지 부분을 읽어 LabWindows/CVI의 기능과 개념에 대해 전반적인 이해를 가진다.
3. 요약된 2~9장까지의 Tutorial sessions을 숙지한다.
4. 함께 배포된 예제 프로그램을 익힌다.

▶Learning LabWindows/CVI

프로그램을 익숙하게 사용하기 위해서 모든 설명서를 완벽하게 읽을 필요는 없다. 초급자들은 Tutorial을 먼저 익히고, Getting Started With LabWindows/CVI에 익숙해졌을 때 User Manual을 읽는 게 좋다. 그리고 이 두 가지를 습득한 후에 나머지 설명서를 읽으면 된다.

▶LabWindows/CVI System Overview

LabWindows/CVI는 C 프로그래머를 위한 소프트웨어 개발 시스템이며, 데이터 취득이나 기계 제어응용을 위한 프로그램과 라이브러리를 개발할 수 있는 대화식 환경(Interactive environment)을 지원한다.

C 프로그램을 편집, 컴파일, 링크, 디버깅하는데 대화식 환경을 사용하게 되며, 여기에는 ***"function panel"***이라는 인터페이스가 쓰이게 된다.

CVI의 성능은 그 Library들에 좌우되는데, 이 Library들은 데이터 취득이나 기계 제어 시스템 개발에 필요한 대부분의 기능들을 가지고 있다.

- Data Acquisition : 7개의 Library (Instrument Library, Data Acquisition Library 등)
- Data Analysis : 3개의 Library
- Data Presentation : User Interface Library
- Networking and interprocess communication applications : 4개의 Library

▶LabWindows/CVI Program Development Overview

모든 언어와 개발 환경에서 공통적으로, 좋은 프로그램을 만들기 위해서는 코드를 작성하기 전에 프로그램의 기능설계가 수행되는 것이 바람직하다. 좋은 기록들을 유지하고 코드에 해설을 다는 것은 프로그램 개발의 향상에 도움을 준다.

<Using C in LabWindows/CVI>

CVI는 계기를 사용하는 기계응용을 위해 C 프로그램을 증진시킨다. 따라서 CVI 작업을 시작하기 전에 C 언어에 대한 기본적인 이해가 있어야한다. 자세한 설명은 Standard Library Reference Manual 의 1장을 참고한다.

그리고 Library Function을 사용할 수 있게 해주는 User Interface 사용방법을 예제를 통해 실행해 볼 수 있다.

<LabWindows/CVI Program structure>

Tutorial을 학습하여 CVI의 예비 단계를 밟은 뒤에는 자기 자신의 작업을 시작해야 되는데, CVI는 융통성이 뛰어나기 때문에 어떤 프로그램을 만들더라도 대부분 접근이 가능하다. 대부분의 프로그램에서 전체는 아니지만 일부분은 다음의 요소들이 반드시 들어가게 된다.

- User Interface
- Program Control
- Data Aquisition
- Data Analysis

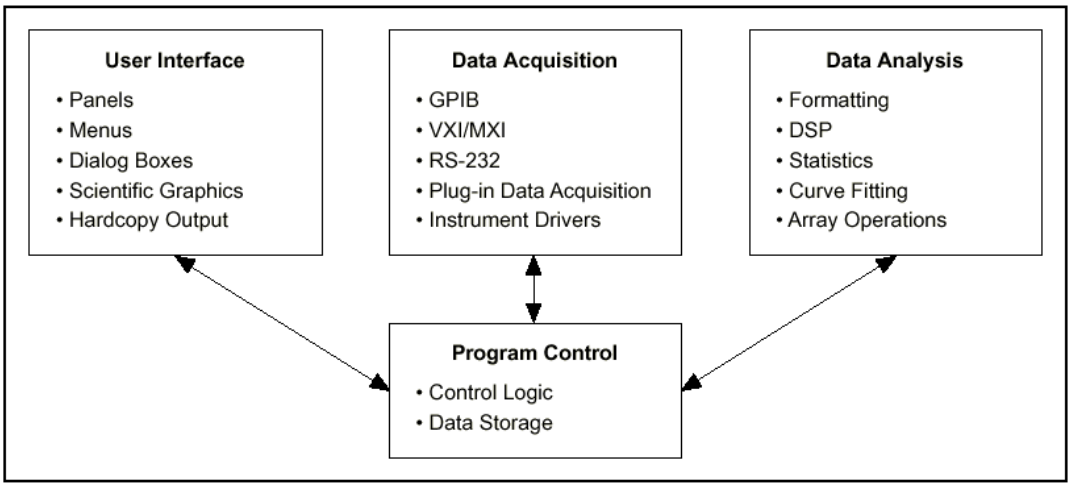


Figure 1-1. Relationship of Program Elements

<User Interface>

CVI User Interface Editor를 사용하면 작은 노력으로도 정교한 Interactive panel들을 만들 수 있다. graphical user interface(GUI)를 설계할 때는 만들려는 프로그램으로 정확히 무엇을 하려는가를 고려하여 결정하여야 한다. User Interface는 menu, panels, controls, dialog box를 통하여 사용자에게 입력과 출력을 제공하는 구조를 가지고 있다.

<Program Control>

Program control 부분은 data acquisition, data analysis, user interface를 조정(통합)하며 사용자 정의 함수와 프로그램 실행 순서 관리를 위한 제어 로직들을 가지고 있다. CVI의 대부분의 프로그램 제어 요소는 사용자가 직접 작성한 코드로 구성된다. 따라서 CVI에 포함되어 있는 예제 프로그램에서 사용된 코드는 사용자가 작성한 프로그램 코드가 어떻게 만들어지는지를 이해하는데 도움을 준다.

<Data Acquisition>

계기로부터 데이터를 얻거나 데이터 취득 보드에 접속하여 데이터를 취득했을 때, 얻은 정보는 약간의 조정이 필요하다. 여기서는 프로그램의 다른 구성요소에 의해 분석되고 전달되어진 초기 데이터(raw data)를 공급하게 된다.

<Data Analysis>

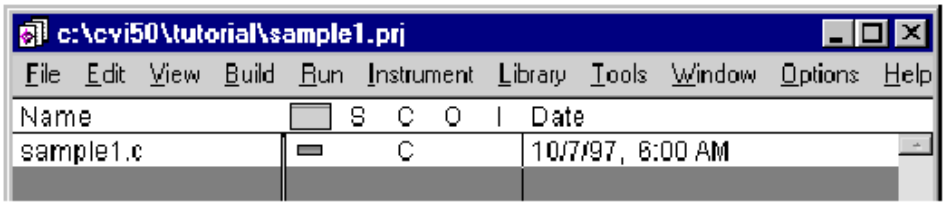
데이터를 얻은 후에는 이것을 분석하는 것이 필요하다. Analysis에는 formatting, scaling, signal processing, statistics, curve fitting등이 포함되어 있는데, Formatting and I/O Library, Analysis Library, Optical Advanced Analysis Library등에 이들 동작을 실행하는 함수들이 들어있다.

2. Loading, Running, and Editing Source Code

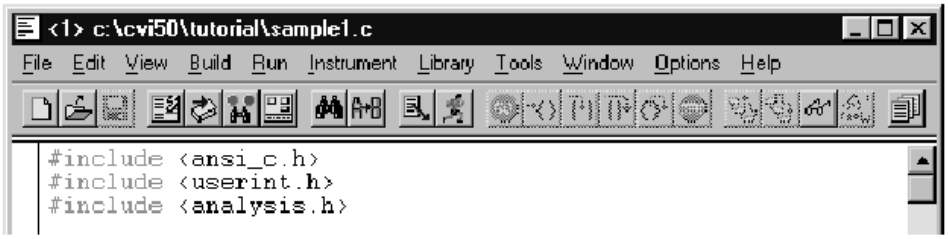
- CVI window 종류
- project 불러오기와 실행
- CVI project에 사용될 수 있는 파일 형식
- CVI에서 가능한 소스 코드 편집기법(source code editing techniques)

▶A Note about LabWindows/CVI Windows

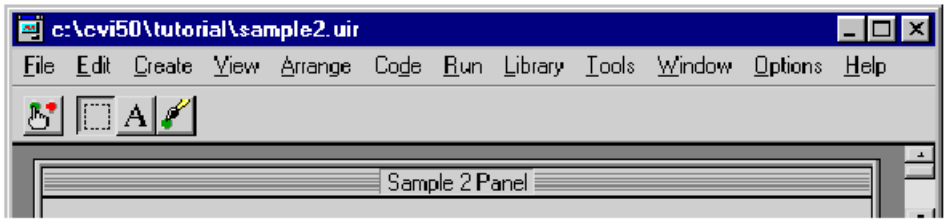
이번 tutorial에서는 Project window, Source window, User Interface window가 사용된다. CVI의 다른 윈도우들과는 다르게 이들 윈도우들은 작업하는 프로그램에 따라 그 제목이 바뀌게 된다. 파일 확장자는 사용자가 지금 어디에서 작업하고 있는지를 알려준다. 예를들어 sample1 project에서 작업하고 있다면 Project 윈도우 맨 위쪽 부분에 .prj 라는 확장자가 나타나며, c:\cvi\tutorial\sample1.prj 라는 제목이 나타나게된다. 마찬가지로 Source window에는 .c의 확장자가, User Interface window에는 .uir 확장자가 나타나게 된다.



<그림 2-1 Project Window>



<그림 2-2 Source Window>

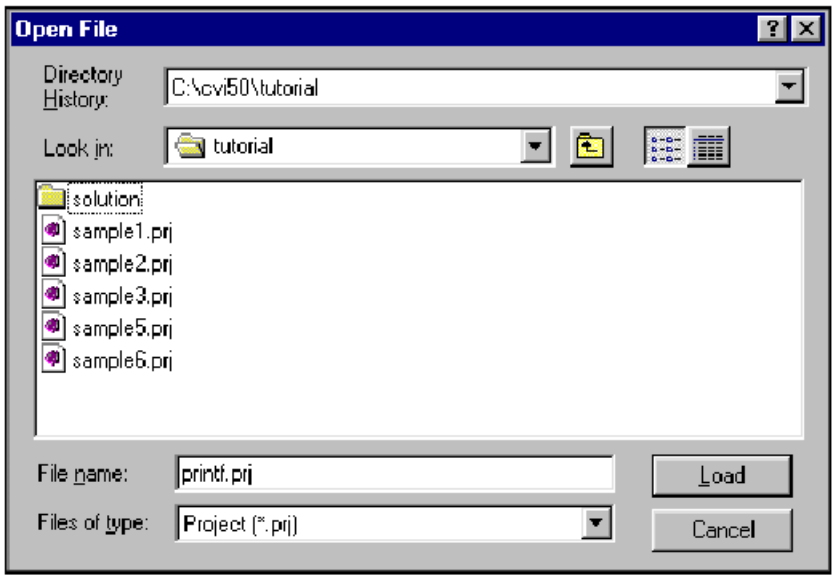


<User Interface Window>

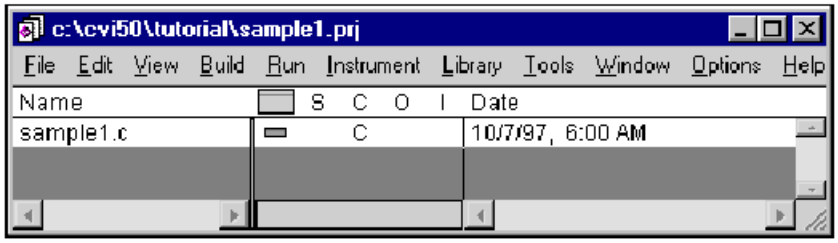
▶Loading a Project into LabWindows/CVI

CVI 편집, 실행 화면을 보기 위해서는 CVI project window로 project를 로드하여야 한다.

1) *File* 메뉴에서 *Open*을 선택하면 CVI에서 제작, 편집 할 수 있는 여러 가지 파일형식들과 함께 list가 나타난다.
2) CVI/tutorial 디렉토리에서 Project(*.prj)를 선택한다.
3) sample1.prj project를 선택한다.



<그림 2-4 Open File Dialog Box>



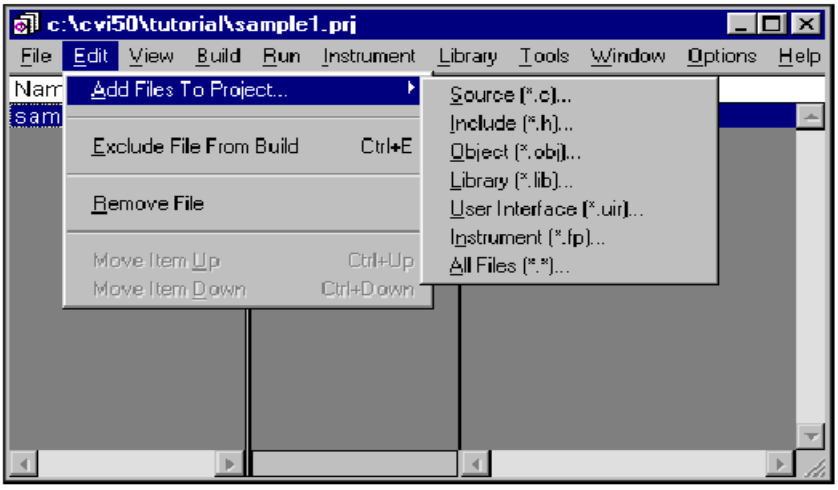
<그림 2-5 sample1 project window>

이제부턴 Project window, standard I/O window, Source window에 대해 각각 살펴본다.

<Project Window>

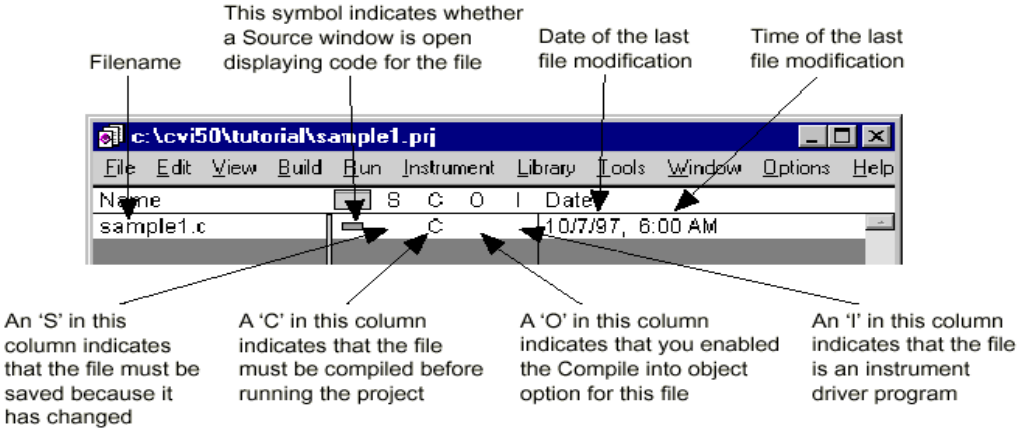
Project window는 특정 project나 프로그램을 구성하는 모든 파일을 보여준다.

CVI로 만든 project는 많은 형식의 복합 파일들로 구성될 수 있다. CVI에는 C source파일과 object module, C library, user interface file, instrument driver등이 같이 공존하므로 추가적으로 project를 개발할 때에는 *Edit* 메뉴의 *Add File to Project*를 통해 원하는 형식의 파일을 추가하면 된다.



<그림 2-6 Adding File to Project>

sample1.prj는 하나의 C source 코드로 구성된 간단한 project이다. project window에는 파일들의 정보가 표시되는데 다음 그림에 이를 나타내었다.



<그림 2-7 Information in the Project Window>

<Running the Project>

sample1 project를 실행하기 위해서 *Run* 메뉴의 *Run Project*를 선택한다. CVI는 자동적으로 컴파일과 링크를 하여 코드를 실행한다. 컴파일 후엔 그림 2-7에서의 C 표시는 사라지게 된다.

sample1은 100개의 난수를 발생하여 Standard I/O window에 출력해 주는 프로그램이다.

<Error Messages>

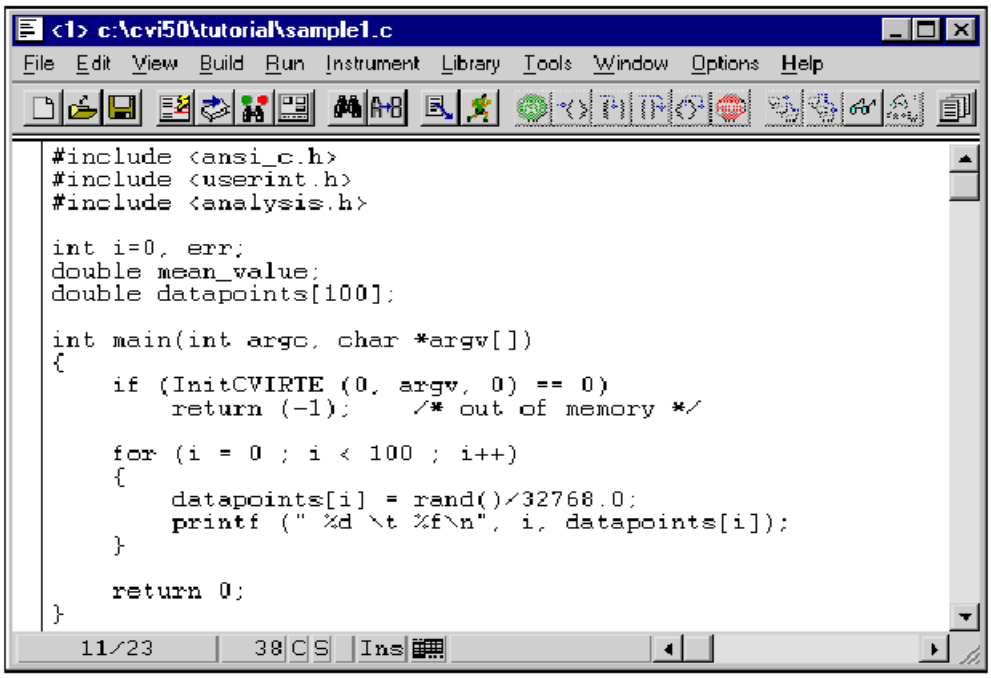
컴파일러가 컴파일이나 링크처리 도중 에러를 발견하면 에러 메시지가 출력된다. 에러 메시지 window는 각 source 파일에서 발견된 에러의 개수와 현재 에러에 대한 설명을 포함하고 있다.

<Standard Input/Output Window>

표준 입출력 창은 간단한 텍스트 모드로 되어있으며 프로그램이 실행되는 동안 사용자로부터 정보를 받거나 보여준다. CVI에서 C 프로그램을 개발할 때 stdio library를 사용한다면 printf나 scanf 함수의 결과가 표준 입출력 창에 나타난다.

<Source Window>

1) *File* 메뉴의 *Hide*를 사용하거나 단축기 <Ctrl-W>를 사용하여Standard I/O window를 닫는다.
2) source code를 source window에 나타내게 하기 위해서 project 목록의 sample.c를 더블클릭 한다.



<그림 2-8 예제1의 Source Code>

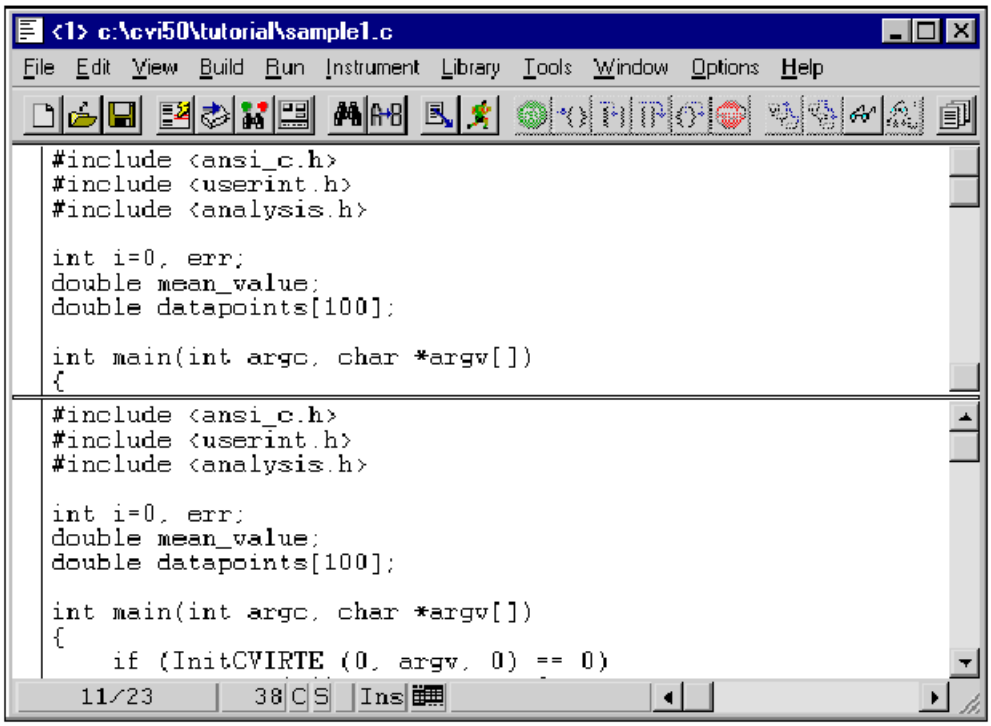
<Editing Tools>

CVI Source window는 용량이 큰 파일을 작업할 때 아주 유용한데, 여러 개의 편집 화면을 사용하여 용량이 큰 source에서도 작업을 원활히 할 수 있다. 다음의 예제는 이들 편집 화면들을 설명하고 있다.

1) 용량이 큰 파일을 볼 때는 특별한 줄 번호가 있으며 편리하다. *View* 메뉴의 *Line Number*를 선택하면 window 왼쪽에 줄 번호를 가진 열이 새로 생긴다.
2) 많은 경우에 CVI로 개발하는 프로그램은 헤더파일이나 user interface 파일 같은 다른 파일들을 참조하게 된다. 이러한 부가적인 파일들은 커서를 source 내의 파일이름에다 위치시키고 *File* 메뉴의 *Open Quoter Text*나 <Ctrl-U>단축키를 사용하여 즉시 볼 수있다. ansi_c.h에 커서를 위치시키고 <Crrl-U>를 누르면 ansi_c.h의 헤더 파일이 표시된다.

   File 메뉴의 Close를 사용하여 헤더파일을 닫고 활성창에 sample1.c가 나타나게 한다.

3) 용량이 큰 소스 파일에서 어느 부분을 작성하고 있다면 같은 파일에서 다른 부분을 변경하려면 subwindows라는 것을 이용하여 분리된 창에서 작업을 할 수 있다.
창을 분리하기 위해서 source window의 맨위 부분의 2중선을 클릭하여 화면 중앙으로 drag한다.



<그림 2-9 Split Source Window>

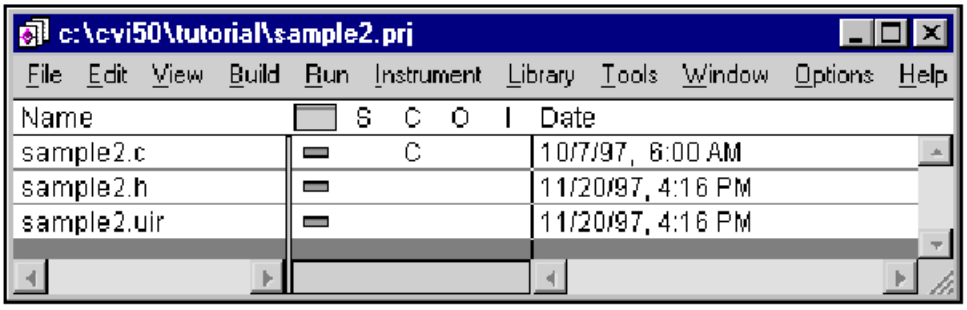
4) 어떻게 각각의 창들이 동기화 되어 다른 영역에서 독립적으로 사용될 수 있는지 예를 들면, 5번 행에 커서를 위치시키고 어떤 문자를 써보면 두 개의 창에 같은 내용이 써짐을 확인할 수 있다.
5) Source window에서 코드를 편집할 때 잘못 입력한 사항은 Undo(Ctrl-Z)를 이용하면 된다. CVI에서는 100번까지 Undo가 가능하다.
6) 다시 2중선을 Drag하여 단일 창으로 복귀시킨다.
7) source code에서 신속하게 특정 행으로 이동하는데는 두 가지 방법이 있다. 원하는 특정 행으로 이동하려면 *View*메뉴의 *Line*(Ctrl-L)을 사용한다.
또 다른 방법은 tag를 사용하는 것인데 커서를 3행에 위치시키고 *View* 메뉴의 *Toggle Tag*를 선택한다. source window의 왼쪽 열에 초록색 사각형이 나타난다.
8) 커서를 12행에 위치시키고 다른 tag를 설정한다. *View* 메뉴에서 *Next Tag*를 선택하면 커서는 자동적으로 선택한 행으로 이동한다. <F2>키를 눌러도 같은 동작을 하게된다.

다음 항목으로 이동하기 전에 sample1을 닫는다.

<Operating Project with a User Interface>

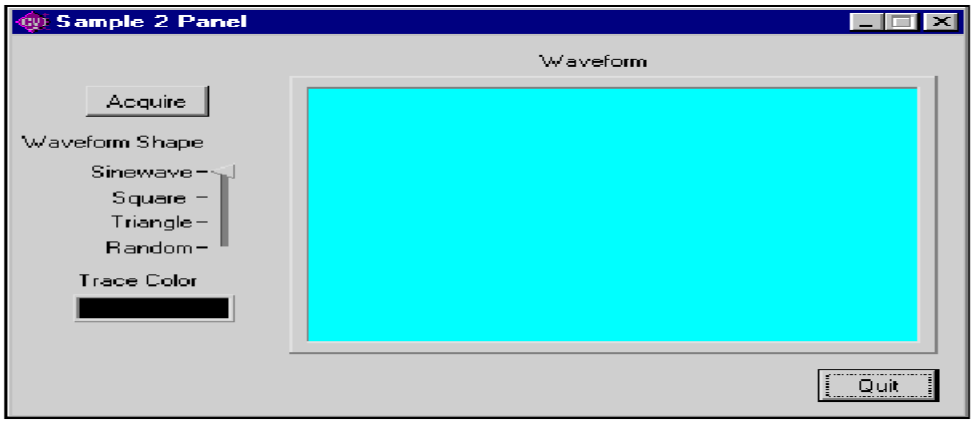
CVI는 Standard I/O window를 통해 아주 간단한 text 기반의 화면을 제공한다. 그러나 대부분의 응용에서 사용자는 graphical user interface(GUI)를 적용하기를 원한다. 5장의 tutorial에서 GUI에서 학습하며 여기서는 간단하게 GUI를 살펴 보기로 한다.

1) *File* 메뉴에 *Open*을 선택하여 sample2.prj를 선택한다. sample1은 저장하지 않고 닫는다. 그러면 다음과 같은 project window가 뜬다.



<그림 2-10 sample2 project window>

2) Run 메뉴에서 Run Project(Shift+F5)를 선택하여 project를 실행시킨다. 프로그램이 컴파일, 실행된 후에 다음과 같은 GUI 화면이 나타난다.



<그림 2-11 sample2.uir 실행화면>

3) GUI화면의 Acquire를 눌러 파형을 화면에 나타나게 한다. 파형이나 색깔등을 바꿔가며 그 변화를 살펴본다.
4) Quit을 사용하여 프로그램 실행을 중지한다.

나머지 장에서 sample2와 비슷한 예제를 실행 해 볼 것이며 CVI를 사용하여 GUI를 설계하고 코드를 생성시키는 것을 배우게 될 것이다. 다음 3장에서는 Interactive Code Generation Tools에 관해 학습한다.

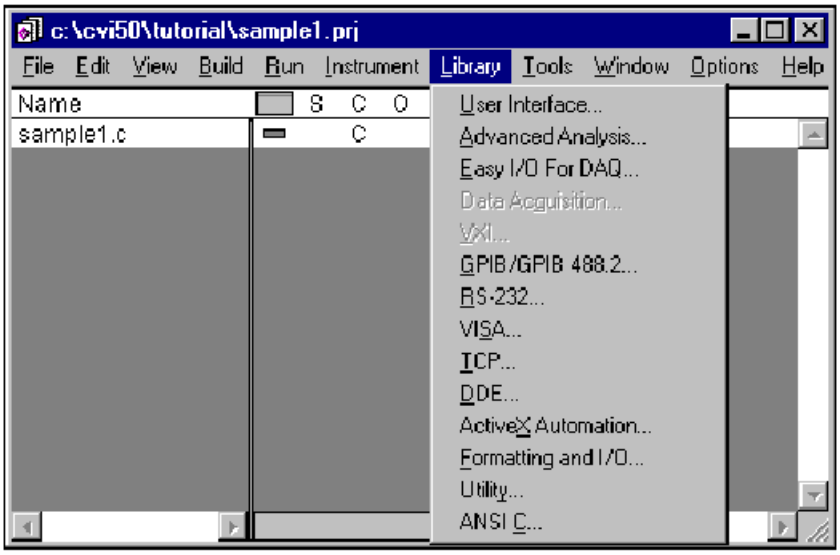
3. Interactive Code Generation Tools

1) project window를 제외한 모든 창을 닫는다.

2) sample1.prj 파일을 로드한다.

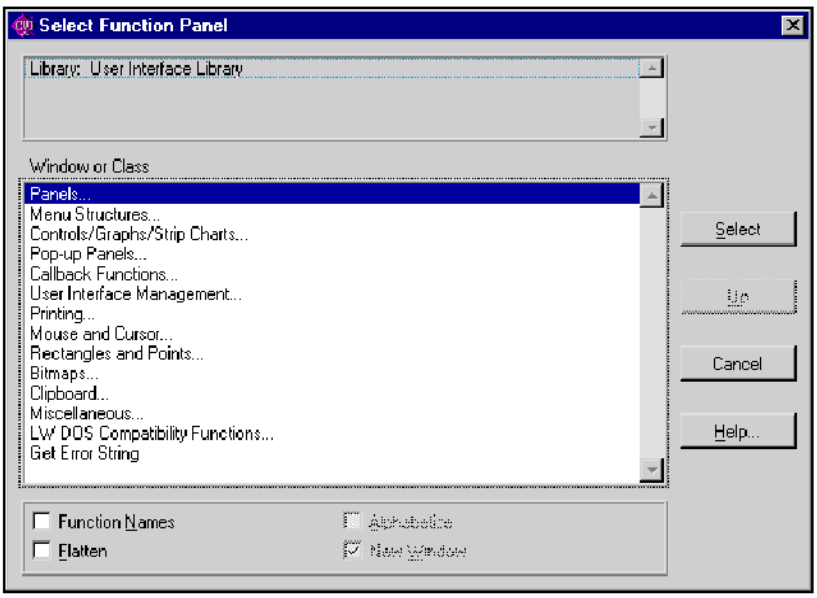
<Library Menu>

CVI의 모든 library들은 다음 그림과 같이 Library 메뉴아래에서 계층적인 구조로 분류되어있다



<그림 3-1 Library Menu>

Library 메뉴에서 하나의 library를 선택하면, function tree가 나타난다. 그림 3-2에 User Interface Library function tree를 나타내었다.



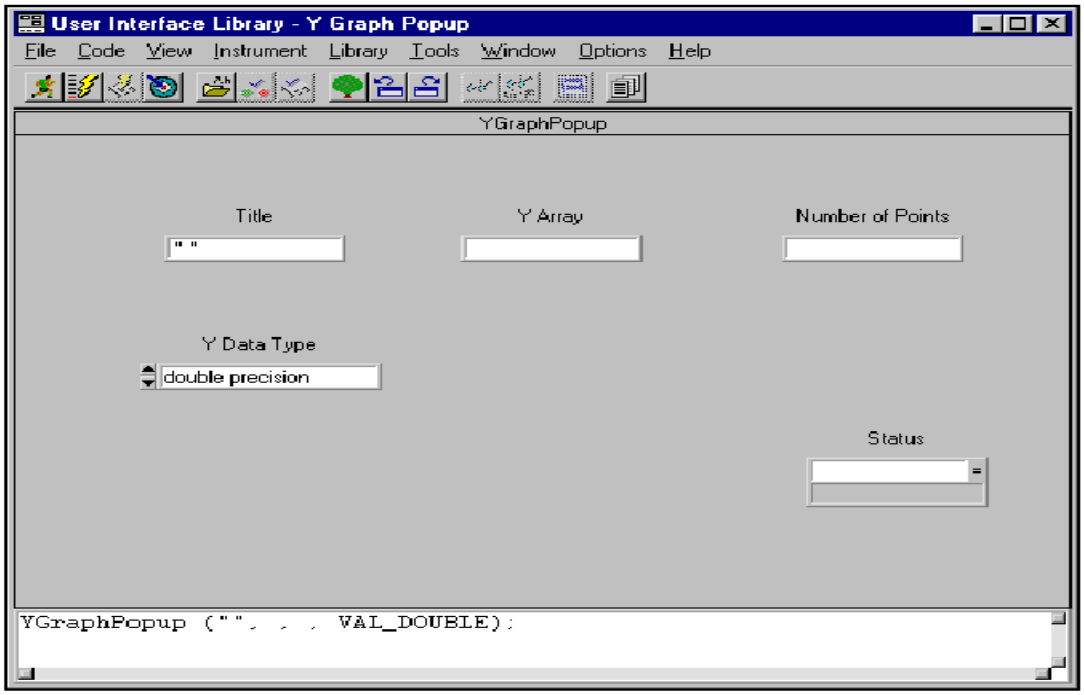
<그림 3-2 User Interface Library Function Tree>

▶Accessing the User Interface Library

sample1 project에서 발생하는 난수를 그래프로 나타내기 위해 User Interface Library를 사용해 보자. 그 전에 앞에서 열어두었던 window는 닫는다.

1) source 코드를 보기 위해 project window에서 sample1.c를 더블 클릭한다.
2) Run 메뉴에서 Run Project를 선택하여 프로그램이 제대로 동작하는지를 확인한다. 정상적이라면 Standard I/O window에 100개의 숫자가 출력되어야 한다.
3) 프로그램 실행이 끝나면 Standard I/O window를 닫는다.
4) 입력 커서를 source의 line 19의 return 0에 놓는다.
5) Library 메뉴의 User Interface를 선택하면 대화 창이 뜬다.
6) P키를 누르거나 화살표키를 사용하여 Pop-up Panels를 선택하면 다른 대화창이 나타난다.
7) Y키를 눌러 YGraphPopup 항을 선택한다.

YGraphPopup 항을 선택한 화면이 그림 3-3에 나타나있다.



<그림 3-3 Y Graph Popup Function Panel>

▶Function Panel Fundamentals

YGraphPopup을 선택한 후에 나타난 화면을 function panel이라 하며 function panel은 CVI library function의 그래픽 화면이다. Function panel은 CVI에서 4가지의 중요한 용도를 제공한다.

1) Funcion panel은 함수호출에서 각 파라메터의 의미와 CVI library의 각 함수의 용도를 알려주는 online help를 가지고 있다.
2) 메모리내의 함수를 자동적으로 선언 할 수 있다.
3) 프로그램과 결합시키기 전에 각 CVI함수를 상호적으로 실행할 수 있다. 여기서 함수의 실행이 만족될 때까지 파라메터 값을 변화시켜 실험해 볼 수 있다.

4) Function panel은 자동적으로 코드를 생성시키며 호출된 함수는 자동적으로 프로그램 소스코드에 삽입된다.

<Function Panel Controls>

function panel의 항들은 파라메터를 지정을 위한 조정장치라 불린다. 여기에는 8가지의 형식이 있는데 다음 예제에서 다룰 것이다.

Input control이라고도 불리는 title control이 선택되며 여기에 숫자 값이나 변수명을 입력할 수 있다.

<Tab>키를 누르면 다른 control들이 선택되어지며 마우스로 선택할 수도 있다.

<Drawing a Graph>

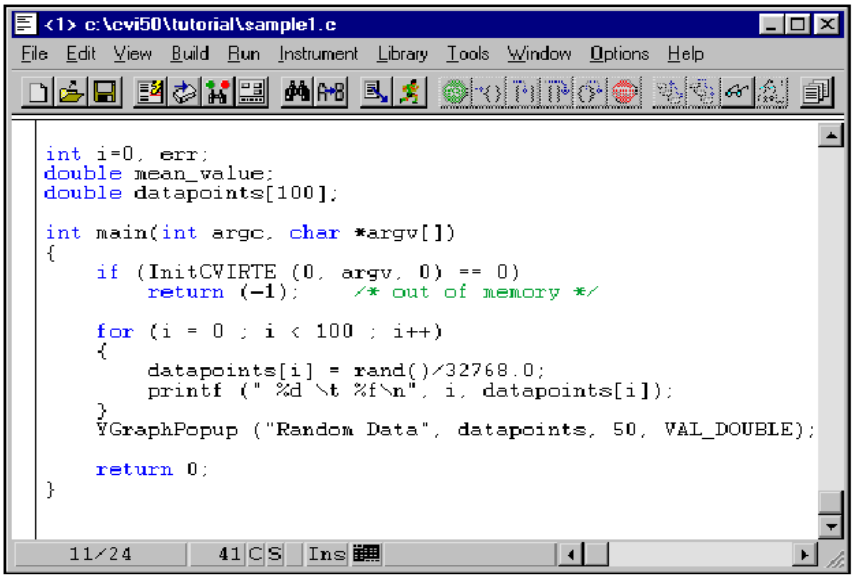
sample1 프로그램에서 만들어진 난수를 그래프로 나타낼 코드를 function panel을 사용하여 작성해 본다.

1) Title control에서 Random data를 “”사이에 입력한다.
2) Y array control에 datapoints를 입력한다.
3) Number of points control에 50을 입력한다.
4) Y data type control에서 double precision을 선택한다.
5) Status control에서 변수명 err을 입력한다.

<Inserting Code from a Function Panel>

Function panel의 하단에는 코드 생성 창이 있다. function panel에서 control을 조정할 때 코드생성 창에서 코드 행이 만들어진다. 응용 프로그램에 이들 코드 행을 직접 복사해 넣을 수 있다.

1) Code 메뉴에서 Set Target File을 선택하여 대화창을 열어 sample1을 선태한다.
2) Code 메뉴에서 Insert Function Call을 선택하여 코드를 삽입한다.
3) YGraphPopup function panel 창을 닫는다.
4) 그림 3-4에서와 같이 프로그램 하단에 YGraphPopup 함수가 추가 되었다.



<그림 3-4 Function panel로부터 코드가 삽입된 Source window>

5) Run 메뉴의 Run project를 실행시킨다. 변화된 소스코드를 저장하고 프로그램을 실행시키면 Standare I/O window에 스크린이 나타나면서 그래프가 표시된다.
6) Standard I/O window를 닫는다.

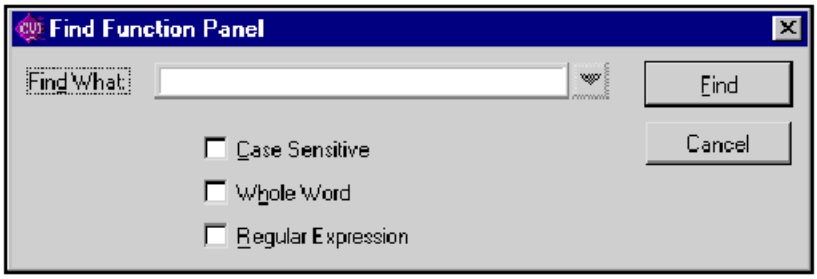
<Analyzing the Data>

커서를 프로그램 window의 다음행에 위치시킨다.

err = YGraphpopup ("random data", datapoints, 50, VAL_DOUBLE)

Mean 함수를 불러 적당한 소스코드 자리에 삽입하기 위해 다음을 실행한다.

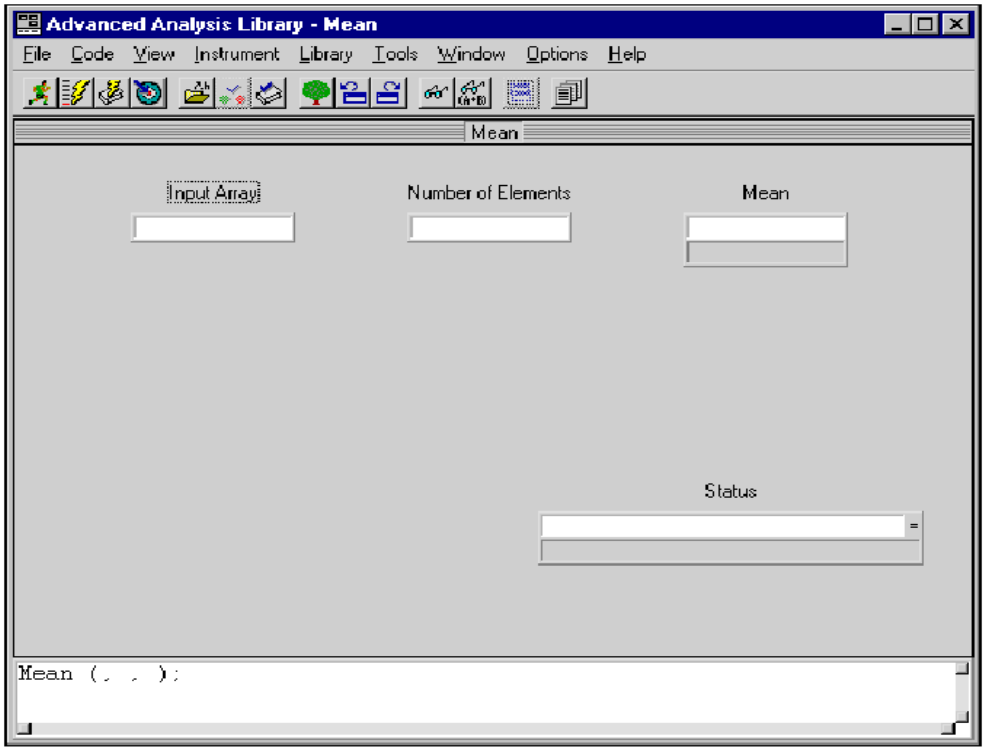
1) View 메뉴의 Find Function Panel을 선택한다.



<그림 3-5 Find Function Panel Dialog box>

2) Function name input box에 Mean을 입력한다.
3) <enter>하여 Mean function panel을 찾는다.

function panel이 그림 3-6과 같이 나타난다.



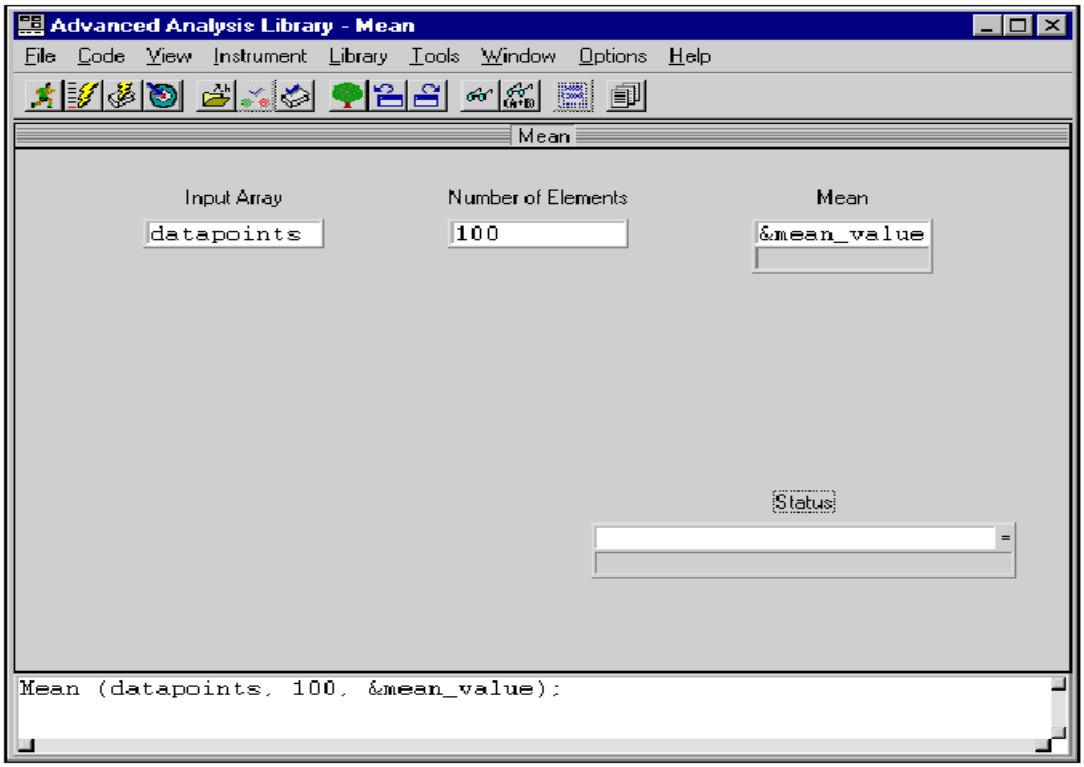
<그림 3-6 Mean Function panel>

1) Input Array control에 datapoints를 입력한다.
2) Number of element control에 100을 입력한다. 나머지 control은 비워둔다

<Output Values on a Function Panel>

Mean function panel의 Mean control은 출력 control이다. 출력 control은 함수를 실행한 결과값을 나타내준다. 결과를 저장할 수 있는 변수명을 지정할 수 있다.

1) Mean control을 선택하여 &mean_value를 입력한다.



<그림 3-7 전체 Mean function panel>

2) Code 메뉴에서 Insert Function File을 선택하여 sample1 소스코드로 Mean 함수를 불러 라인을 삽입한다.
3) File 메뉴에서 Close를 실행하면 source window에서 커서가 있던 위치에 코드가 삽입된 것을 볼 수 있다.

<Recalling a Function Panel>

100개의 요소로 된 배열을 그리기 위해서는 코드를 수정해야 하는데, Program window에서 직접 수정하거나 TGraphpopup function에 관련된 함수를 바꿈으로서 변환가능하다.

1) 커서를 프로그램 창의 다음 항에 위치시킨다.
   err = YGraphPopup ("Random Data", datapoints, 50, VAL_DOUBLE)
2) View 메뉴에서 Recall Function Panel을 선택한다. YGraphpopup function panel이 나타난다. control은 Program window의 code 상태에 따라 자동적으로 변하게된다.
3) Number of point를 선택하여 100을 입력한다.

4) Code 메뉴에서 Insert function Call을 선택하여 프로그램에 새로운 코드를 복사한다. Replace를 선택하여 새롭게 작성된 코드로 수정시킨다.

5) Program window로 돌아와서 YGraphPopup 이 100으로 바뀐 것을 확인한다.

끝으로 다음 행을 메인 함수의 끝부분에 추가시킨다.

printf ("Mean = \t %f \n", mean_value)

Run 메뉴에서 project를 실행시킨다.

다음은 최종완성된 프로그램이다.

```
    printf ("Mean = \t %f \n", mean_value);
```

The completed program listing is shown in the following illustration.

```
#include <ansi_c.h>
#include <userint.h>
#include <analysis.h>

int i = 0, err;
double mean_value;
double datapoints[100];

int main(int argc, char *argv[])
{
    if (InitCVIRTE (0, argv, 0) == 0)
        return (-1); /* out of memory */

    for (i = 0 ; i < 100 ; i++)
    {
        datapoints[i] = rand()/32768.0;
        printf ("%d \t %f\n", i, datapoints[i]);
    }

    YGraphPopup ("random data", datapoints, 100, VAL_DOUBLE);
    err = Mean (datapoints, 100, &mean_value);
    printf("Mean = \t %f \n", mean_value);

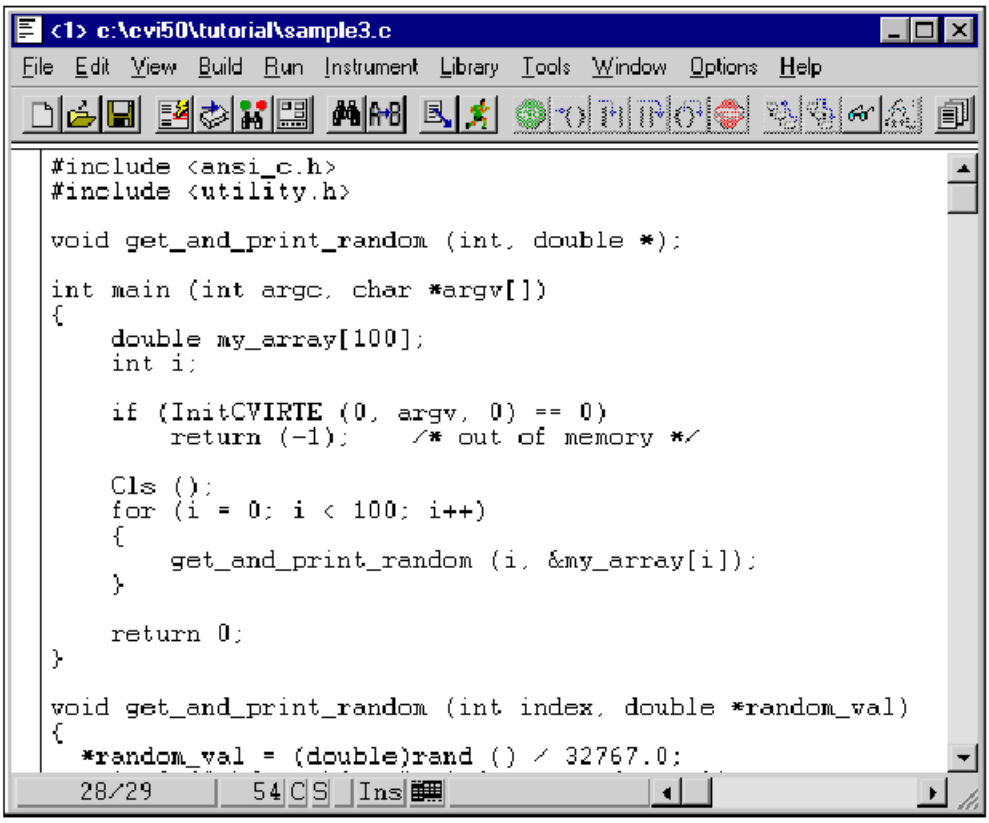
    return 0;
}
```

4. Executing and Debugging Tools

이 장에서는 CVI에서 실행과 디버깅에 도움을 주는 몇가지 도구들을 알아본다. Step mode execution, breakpoint, Variables window, Array Display, String Display, Watch window

▶ Setting up

1) sample3 project를 연다.
2) project window를 제외한 모든 창을 닫는다.
3) sample3를 더블 클릭하여 source window를 연다.



<그림 4-1 sample3.c program>

sample3는 앞의 sample1과 같은 난수를 발생시키지만 다른점은 난수 지정을 단계적으로 할 수 있고 그 값들을 출력할 수 있는 점이 다르다.

▶Step Mode Execution

단계별 실행은 디버깅 프로그램에서 유용한 툴이다.

1) 첫 번째 라인에서 실행을 정지시키기 위해 Run 메뉴에서 Break at First Statement를 선택한다. Break at First Statement 모드가 활성화 되면 √ 가 표시된다.
2) Run Project를 선택하여 프로그램을 실행시킨다. 프로그램이 컴파일 된 후 프로그램의 main 함수라인에 강조선이 나타나며 프로그램 실행이 대기하게 된다.

Run 메뉴의 Terminate Execution을 선택하거나 Terminate Execution icon을 선택하여 프로그램을 정지시킬 수 있는데, 지금은 정지시키지 않도록 한다.

3) 강조된 라인을 실행시키기 위해 Run 메뉴의 Step Into를 선택하거나 Step Into icon을 선택할 수 있다. 단축키는 <F8>을 사용하면 된다. 다른 단계별 명령은 Step Over<F10>, Continue<F5>가 있다. Step Over은 실행되는 함수코드를 단일 단계로 실행시키는 Step Into와는 반대로 함수 코드 자체에서만 함수 호출을 실행한다. Continue는 프로그램이 breakpoint에 이르거나 완전히 실행될 때까지 동작한다.
4) sample3.c의 get_and_print_data 함수와 같이 CVI에서 사용자 정의 변수나 함수를 사용하여 작업을 할 때는 언제나 Go To Definition icon을 사용하여 함수의 정의를 즉각적으로 위치시킬 수 있다. 즉각 get_and_print_data 함수의 정의를 찾을 때에는 sample3,c의 17 행을 더블 클릭하여 강조선을 만든 후, Edit 메뉴의 Go To Definition이나 Go To Definition icon을 사용하면 된다.
   이 기능은 project window에 열거된 소스파일 뿐 아니라 다른 소스파일에 존재하는 함수에도 유효하다. 이 명령을 사용하여 변수 선언도 찾을수가 있다. Go To Definition icon을 사용하여 sample3.c에서 변수의 선언문을 찾아보자.
5) 프로그램을 단계별로 실행하기 위해 Step Into를 사용해 보자.
   get_and_print_random이 실행되었을 때 강조선이 함수로 이동하여 내부로 명령을 추적하는 것을 볼 수 있다.

▶Breakpoints

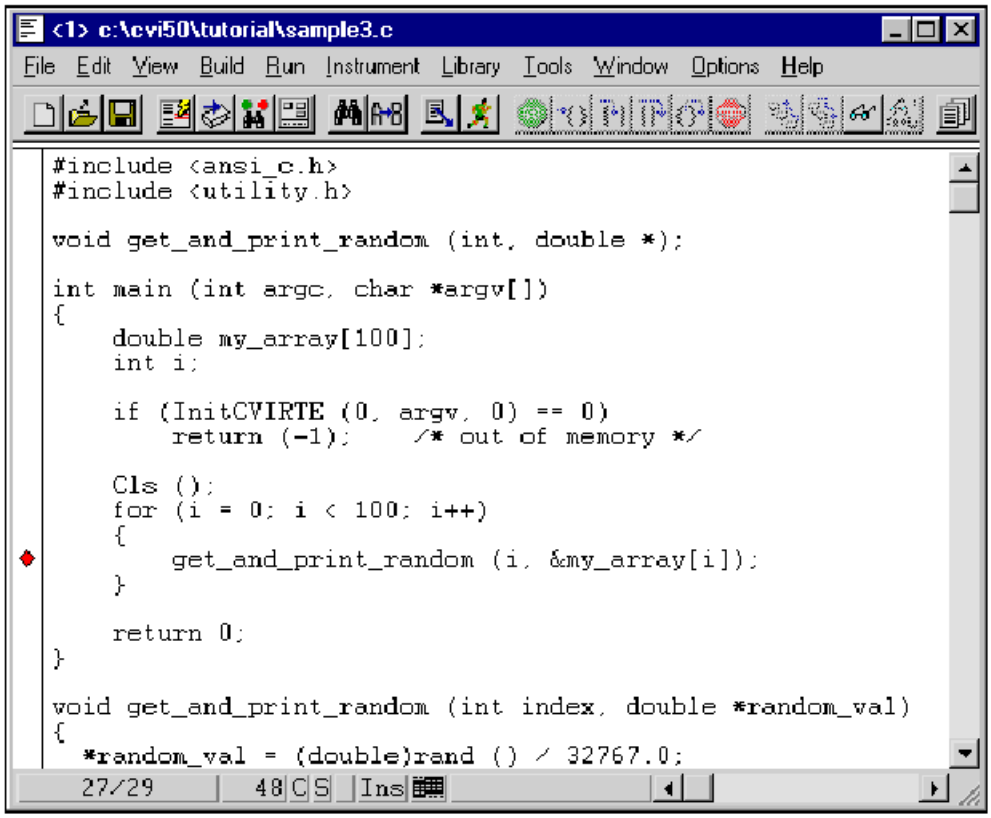
4가지의 Breakpoint 방법이 있다

· Programmatic Breakpoint : 소스코드에서 Breakpoint icon을 삽입.
· Manual Breakpoint : <Ctrl-F12>를 사용한다.
· Breakpoint on error : CVI library 함수가 에러를 발생할 때.
· Conditional Breakpoint : 사용자 지정 조건이 만족되었을 때 .

<Programmatic Breakpoint>

1) 프로그램 실행을 정지시킨다. : Terminate Execution
2) Run 메뉴에서 Break at First Statement를 Disable 시킨다.
3) 소스 창의 get_and_print_random (&i, &my_array[i]) ; 행의 왼쪽 열에 마우스로 왼쪽을 클릭한다.
4) 다음 그림에서 처럼 빨간 다이아몬드 표시가 컬럼에 나타난다.

breakpoint가 어떻게 동작하는지 보기위해 project를 실행시킨다. 프로그램이 동작되는 동안 breakpoint를 만나면 프로그램 실행이 중지되고 빨간 box로 강조되어 행이 나타나게 된다.



<그림 4-2 코드 라인 옆의 Breakpoint>

<F5>를 누르면 프로그램 실행이 계속된다. breakpoint를 만나거나 완전히 실행이 끝날 때까지 실행을 계속하게된다. breakpoint에서 프로그램을 중지시키고 싶으면 <Ctrl-F12>를 사용하면 된다. breakpoint를 프로그램에서 해제하려면 마우스로 다시 한번 다이아몬드 표시를 클릭하면 된다.

<Manual Breakpoint>

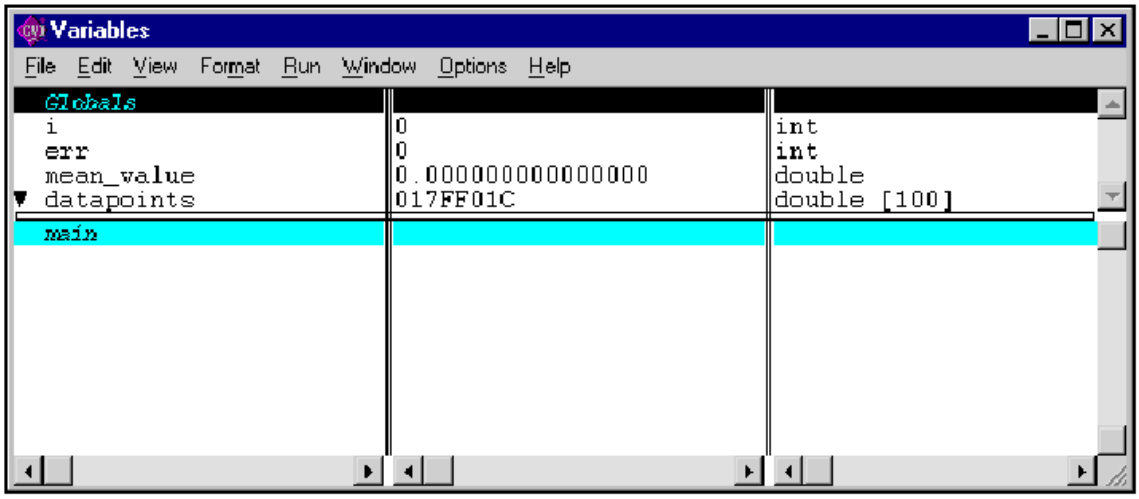
프로그램이 실행되기 전에 <Ctrl-F12>를 이용하여 breakpoint를 입력할 수 있다. 프로그램을 실행시키고 프로그램이 실행되었을 때 <Ctrl-F12>를 누른다. 프로그램은 breakpoint와 만났을 때처럼 breakpoint 모드로 들어가게 된다. 한가지 다른점은 breakpoint를 가진 라인이 강조되는 대신에 프로그램의 다음 실행문이 강조된다는 점이다.

▶Displaying and Editing Data

많은 경우에 프로그램을 test하기 위해서는 소스코드를 보다더 심층적으로 볼 필요가 있는데 CVI에서는 data를 보고 편집하는 특별한 화면을 제공한다. 이것은 Variables window, Array Display, String Display이다. 게다가 CVI는 Watch window라 하는 유틸리티를 제공하는데, 프로그램 실행동안 변수와 값을 보여준다.

<Variable Window>

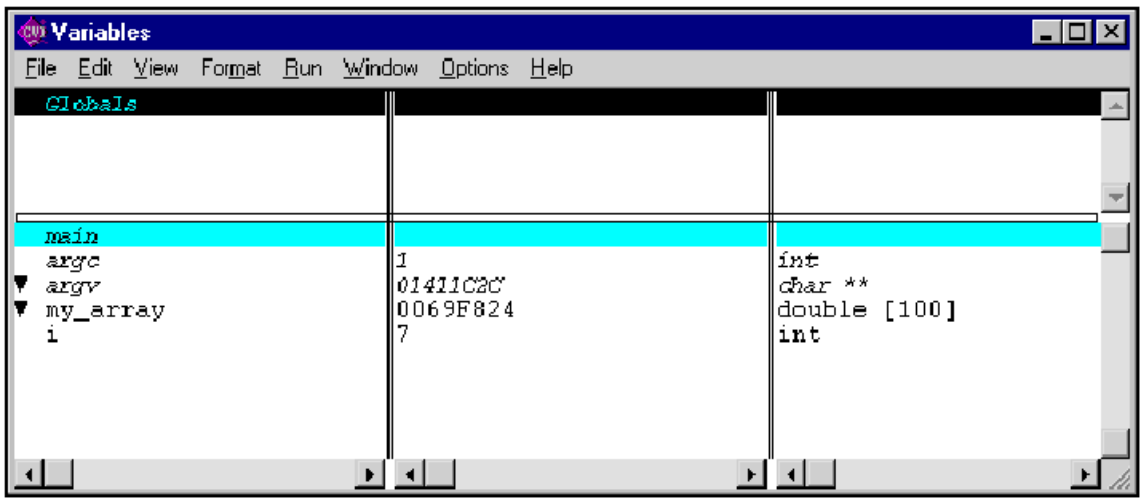
CVI에서 선언된 현재의 모든 변수들을 Variable window에서 볼 수 있다. Window 메뉴에서 Variable을 선택한다.



<그림 4-3 Variable window>

Variable window에는 현재 선언된 각 변수들의 형식과 이름, 값이 나타나있다. 변수들은 그들이 전역, 지역등의 정의를 어떻게 하는가에 따라 분류되어 나타난다. 변수값을 검사하기 위해 언제든지 Variables window를 볼 수 있다. 이것은 특히 breakpoint 실행을 하면서 프로그램을 단계적으로 실행할 때 유용하다.

1) Run 메뉴에서 Break at First Statement를 선택한다. √ 마크가 생길 것이다.
2) 프로그램을 Run 시키면 main 함수가 강조된다.
3) Window 메뉴에서 Variable을 선택한다.



<그림 4-4 Main을 실행할 때의 변수 window>

4) File 메뉴에서 Hide를 선택하거나 Close를 클릭하여 소스 창으로 돌아간다.
5) Run 메뉴의 Step Into를 선택<F8>하여 강조선이 *random_val = (double)rand()/32767.0 라인에 올 때까지 반복한다.
6) Window 메뉴에서 Variable을 선택한다.

새로운 항목이 get_and_print_random 함수 제목밑에 나타난다.

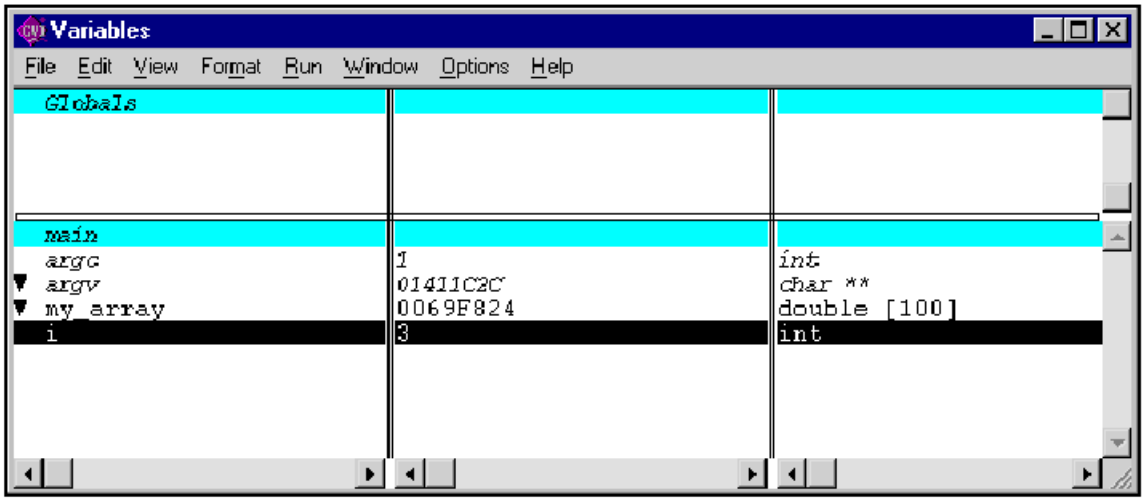
<Edting Variables>

변수관찰에 더하여 변수 편집도 할 수 있다. 프로그램은 아직 breakpoint 모드에서 get_and_print_random 함수안의 *random_val = (double)rand()/32767.0 에 강조되어 있을 것이다.

1) Step Into<F8>을 선택하여 loop를 계속 실행시킨다. 조금뒤에는 강조점이 다음을 가르킬 것이다.

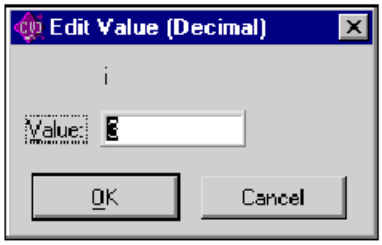
   get_and_print_random (&i, &my_array[i]);

2) i 변수를 선택하고 View Variable Value icon을 클릭하거나 <Shift+F7>을 사용하면 다음 그림과 같이 자동적으로 i 변수가 강조된 Variable window가 나타난다.



<그림 4-5 i 변수가 강조된 변수 창>

3) <Enter>를 누르면 다음 그림에서처럼 편집 값 dialog box가 나타난다.



<그림 4-6 편집 값 dialog box>

4) box에 10을 입력하고 <Enter>
5) 백그라운드의 소스 창을 클릭하여 활성화 시킨다.
6) <F8>로 Step Into를 진행시킨다.
7) Standard I/O output 창을 활성화시킨다.

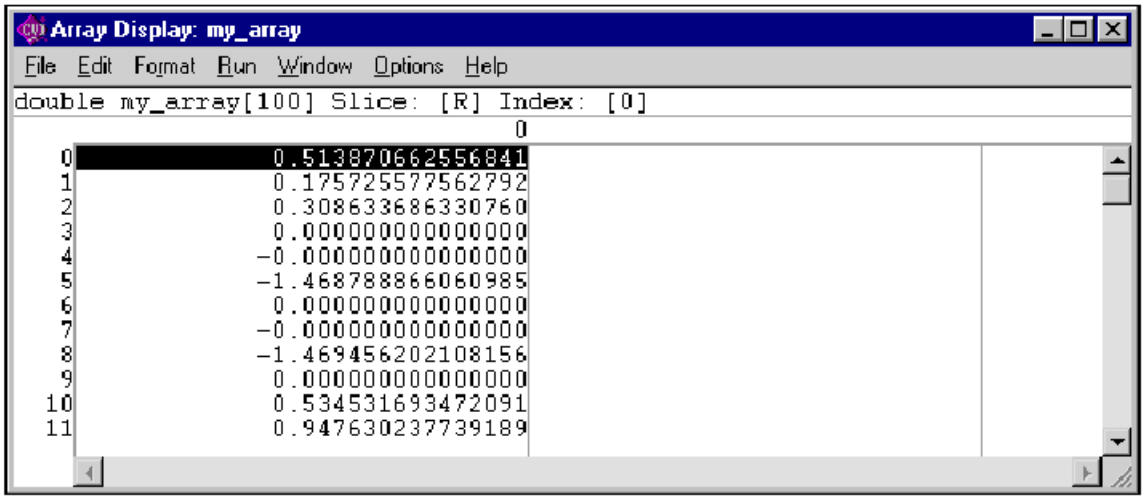
다음 난수가 표시될 때에는 index는 10이된다.

<The Array display>

또 다른 유용한 data display는 Array display이다. Array Display는 data 배열의 내용을 표시한다. 또한 Variables window를 사용하여 변수를 편집했던 것과 같은 방법으로 배열 요소들을 편집할 수 있다.

여전히 breakpoint 모드인 상태에서 다음을 실행한다.

1) 소스 창을 활성화 시킨다.
2) <F8>로 Step Into를 실행시켜 다음항이 강조되게 한다.
   printf (" &d \t %f \n", *index, *random_val);
3) Variables window를 활성화 시킨다.
4) 변수 창의 my_array를 클릭하면 다음과 같은 창이 나타난다.

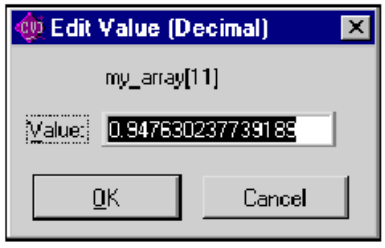


<그림 4-7 Array Display>

<Editing Arrays>

또한 Variable window를 사용하여 개별적인 배열 요소들을 편집할 수 있다.

1) 12번째 요소를 선택하면 다음 창이 나타난다.

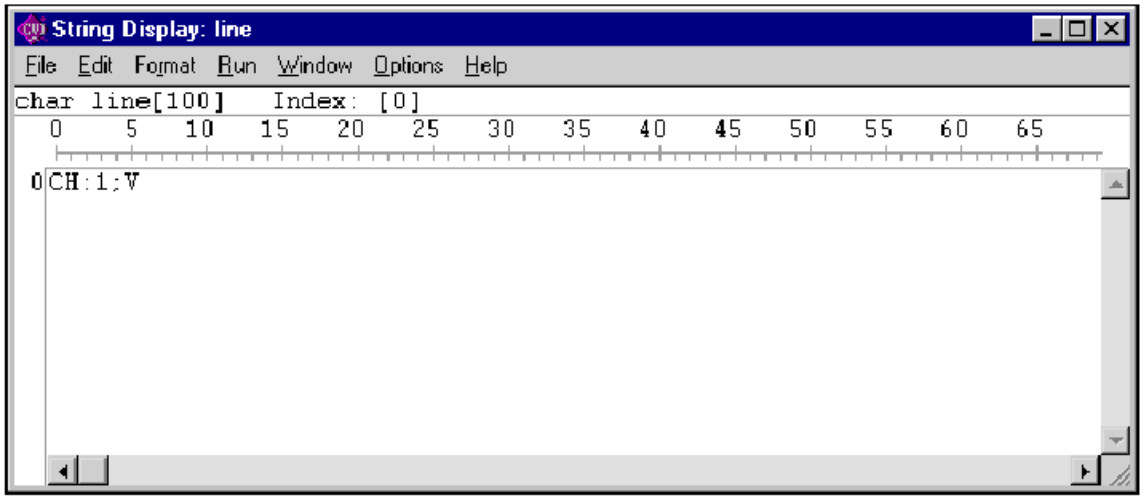


<그림 4-8 Edit value dialog box>

2) 0.5를 입력한다.
3) Array window를 닫고 Variables window로 돌아간다.
4) 소스 창을 활성화 하여 <F8>로 Step Into을 실행한다.

<The String Display>

또 다른 유용한 data 표시는 String Display이다. Variable window에서 string variable을 선택하면 문자열을 입력할 수 있다. 자세한 것은 user manual의 7장에서 다룬다.

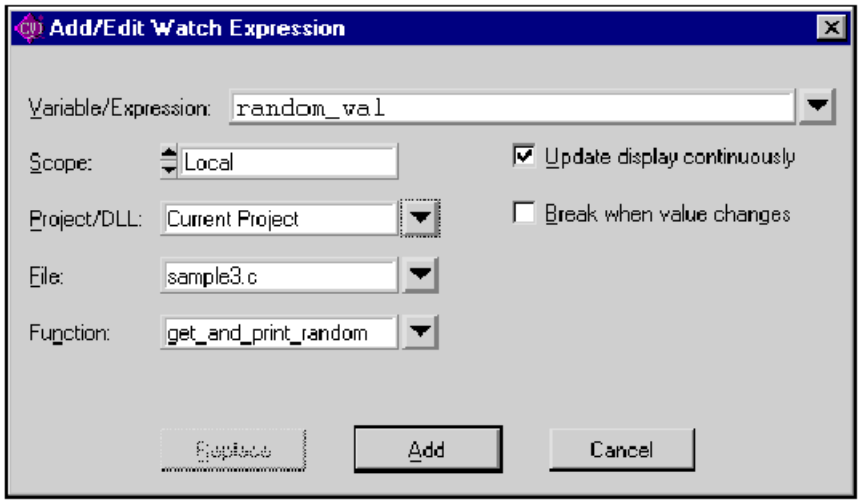


<그림 4-9 String Display>

<The Watch Window>

Watch window는 강력한 디버깅 툴인데, 왜냐하면 프로그램을 실행하면서 시시각각 변화하는 변수 값을 볼 수 있기 때문이다.

1) sample3에서 Break at first statement를 선택한다.
2) <Shift+F5>로 프로그램을 실행시키면 main 함수에 강조점이 나타날 것이다.
3) <F8>로 다음 항까지 프로그램을 실행시킨다.
   void get_and_print_random (index. random_val);
4) Window 메뉴에서 Variables를 선택하거나 백그라운드의 Variables window를 활성화 시킨다.
5) random_val 변수를 클릭한다.
6) Option 메뉴에서 Add Watch Expression을 선택한다



<그림 4-10 Add/Edit Watch Expression dialog Box>

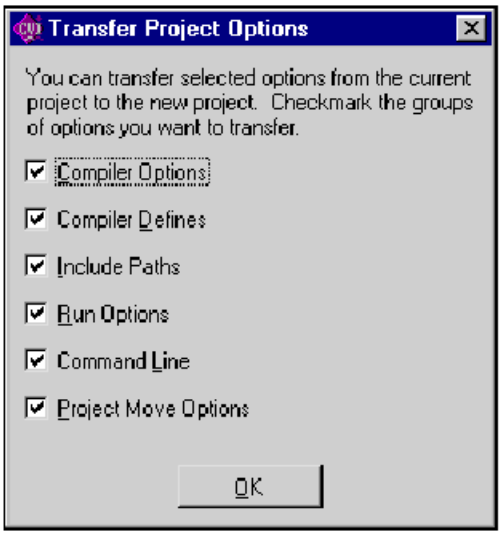
7) Updata Display Continuously에 체크를 한다.
8) 소스 창을 활성화 한 뒤 Watch 윈도의 크기와 위치를 잘 조정한다.
9) Run 메뉴의 Continue를 선택하여 프로그램을 계속 실행시키면 Watch window에 random_val 변수값이 시시각각 변화는 것을 볼 수 있다.

Part II : Building an Application in LabWindows/CVI

5. Building a Graphical User Interface

▶Setting up

1) File 메뉴에서 New를 선택하여 sample3.prj를 unload한다. 새로운 project에 현재의 환경을 그대로 적용할려면 다음에 나타나는 dialog box를 선택하여 OK를 클릭한다.



<그림 5-1 Transfer Project Option Dialog Box>

2) project window를 제외한 창을 모두 닫는다.

<The User Interface Editor>

User Interface Editor는 일반적인 GUI를 만들 때 Drop_and_drag 기능을 사용한다. Create 메뉴에서 여러 가지의 control들을 선택하여 panel에 위치시킬수 있다. 각 control들은 dialog box 시리즈로 원하는데로 setting 할 수 있다.

<Source Code Connection>

User Interface Editor window에서 user interface를 설계한 후 GUI를 제어하기위한 C 소스를 입력할 수 있다. GUI 의 control들을 구분하기위해 user interface에서 각

panel, menu, control에 이름을 지정해야한다. 또한 함수명도 지정해주어야 한다. 특별한 control과 관련된 상수명이나 callback 함수도 지정이 되어야 한다.

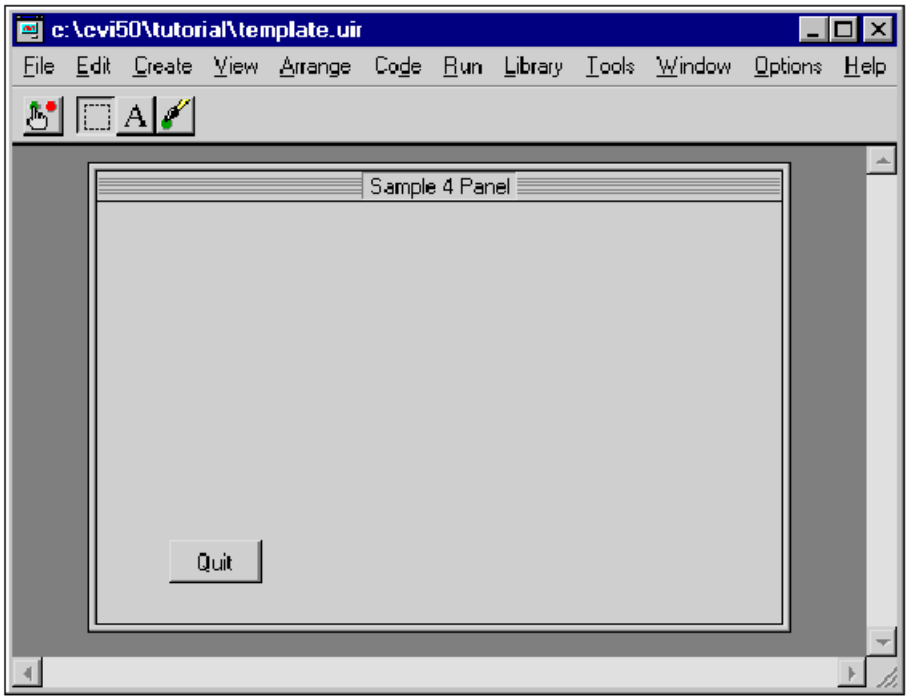
▶The Sample Project

1) User Interface Editor에서 user interface를 생성한다.
2) CodeBuilder를 사용하여 shell program을 만든다.
3) 파형을 생성하고 나타내기 위해 C 소스코드를 삽입한다.
4) 파형의 평균값을 계산하기 위해 Callback 함수를 만든다.
5) data acquisition 시뮬레이터를 위해 project의 장치 드라이버의 사용을 구체화한다.

<Building a User Interface Resource(.UIR) File>

Step 1 : Opening a .UIR File

1) 파일메뉴에서 Open User Interface를 선택한다. template.uir을 선택한다. 다음 그림과 같이 User Interface Editor 화면이 나타난다.

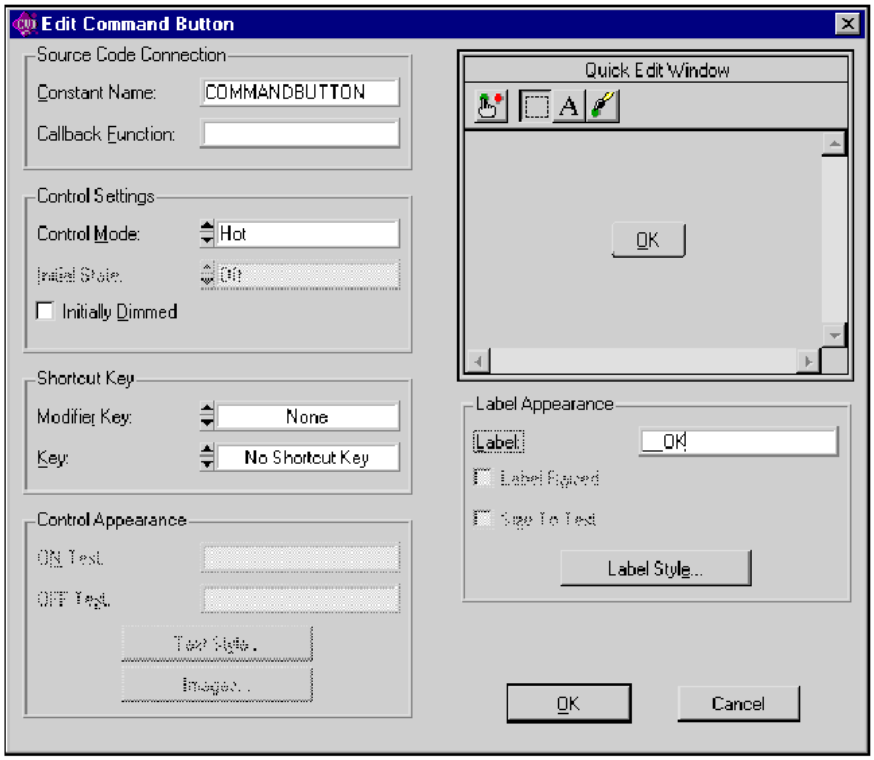


<그림 5-2 User Interface Editor>

2) 그림에서 보는바와 같이 user interface의 일부는 이미 만들어져있고 여기에 acquisition과 동기되는 버튼을 추가하거나 graph control을 추가하면 된다.

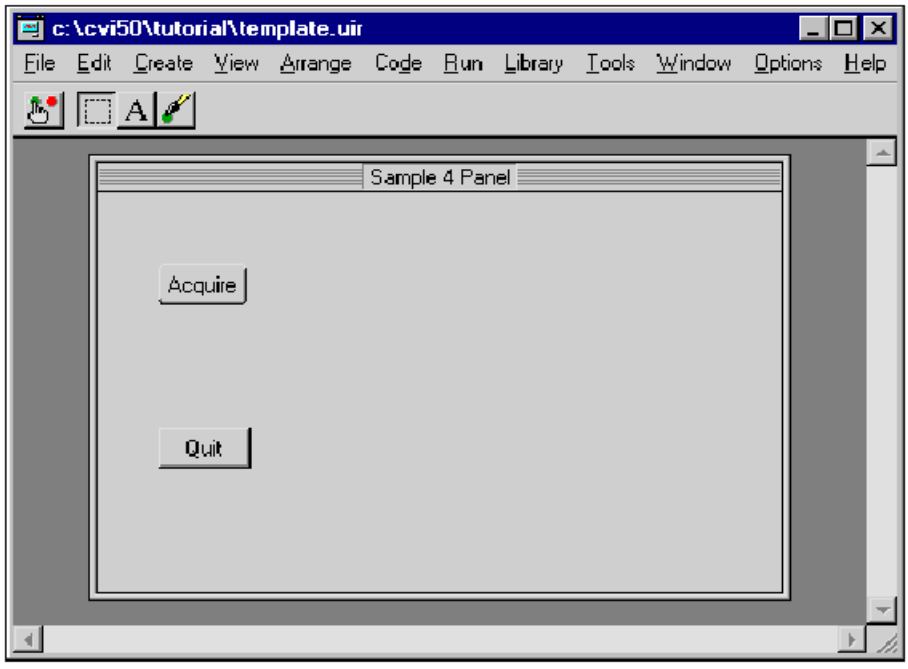
Step 2 : Adding a Command Button

1) Create 메뉴에서 Command Button을 선택하고 메뉴에서 버튼 형식을 선택한다.
2) 버튼 특성을 편집하기위해 OK버튼을 더블클릭한다. 다음과 같은 화면이 나타날 것이다.



<그림 5-3 Edit Command Button Box>

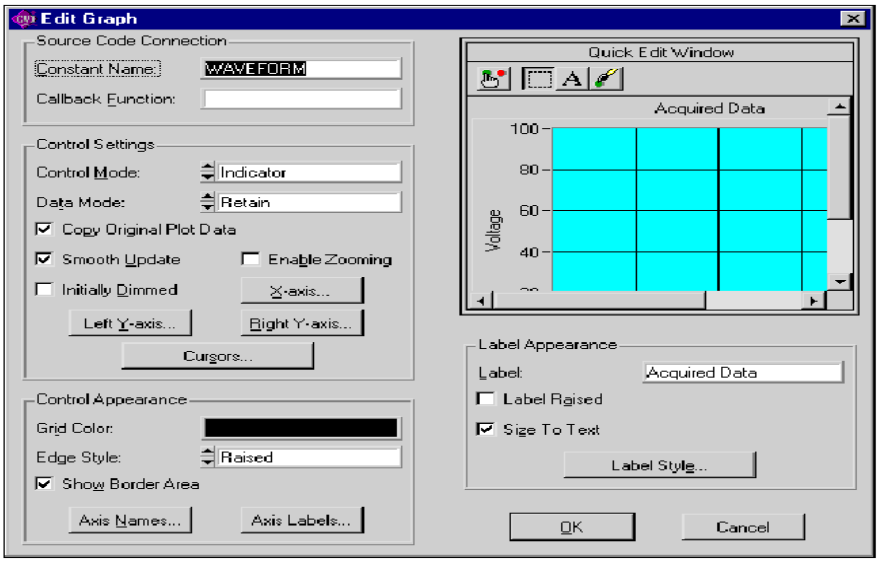
3) 먼저 버튼의 상수명을 지정해야한다. C 소스코드와 연계하기 위해선 이 상수명을 사용해야한다. Source code connection 부분에 ACQUIRE를 입력한다.
4) 다음은 Acquire 버튼을 클릭했을 때 함수명을 지정한다. Callback function input에 AcquireData를 입력한다.
5) Label input 부분에 Acquire을 입력하여 command 버튼의 이름을 바꾼다.
6) OK를 클릭하면 다음과 같은 화면이 나타날 것이다.



<그림 5-4 두 개의 버튼으로 나타낸 User interface>

Step 3 : Adding a Graph Control to the User Interface

1) Create 메뉴를 풀다운하여 Graph 메뉴에서 Graph를 선택한다.
2) 마우스로 graph위치를 잘 조정한다.
3) 그래프 특성을 편집하기 위해 그래프를 더블클릭 한다.
4) Constant Name input에 WAVEFORM을 입력한다.
5) Label Appearance 부분의 Label input에 Acquire Data를 입력한다.

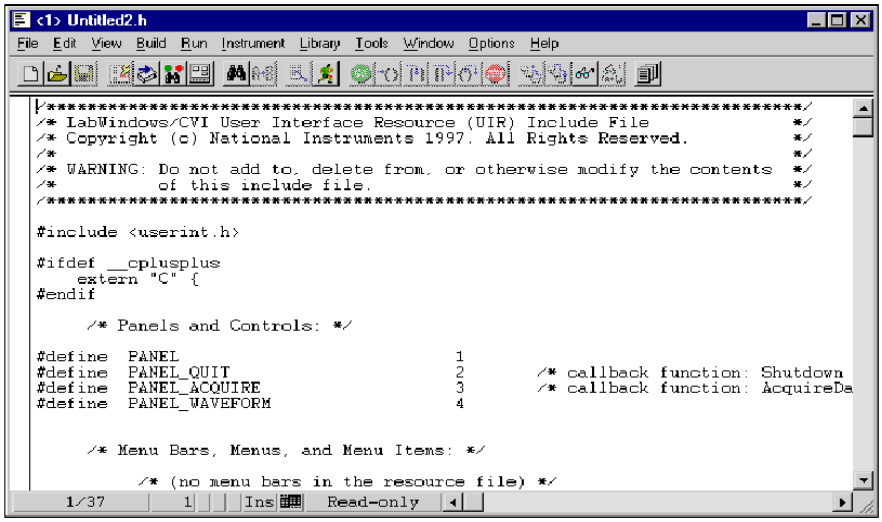


<그림 5-5 Edit Graph Dialog Box>

6) OK를 클릭하여 Edit Graph Dialog box를 닫는다.

Step 4 : Saving the .uir File

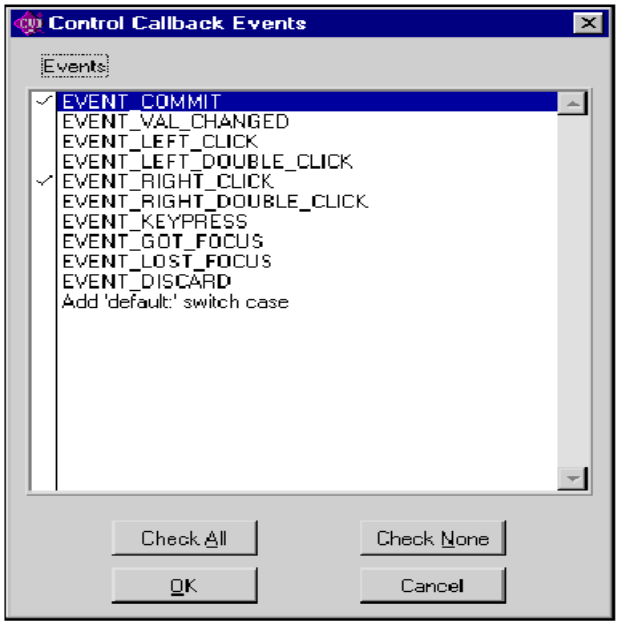
1) 파일 메뉴에서 새로운 control을 가진 .uir을 저장하기 위해 Save as를 선택한다.
2) sample4를 File name input에 입력하고 Save 버튼을 누른다.
3) View 메뉴에서 Preview User Interface Header File을 선택하여 자동으로 생성된 header file을 확인한다.



<그림 5-6 User Interface Header File>

Step 5 : Generating the Program Shell with CodeBuilder

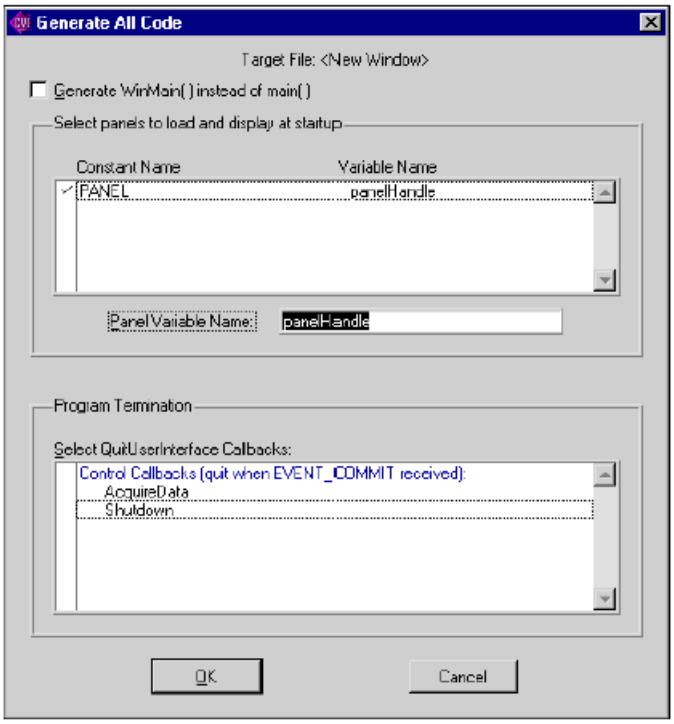
1) 먼저 프로그램이 응답할 상황을 지정해야 한다. Code 메뉴의 Preference로부터 Default Control Events를 선택한다.



<그림 5-7 Control Callback Events Dialog Box>

2) 이 예제는 최종적으로 사용자가 GUI control에서 right-clic 했을 때 help information이 나타나게 되어있다. 이를 위해서 control callback events에서 EVENT_RIGHT_CLICK을 선택한다. 프로그램은 두 가지 상황에 응답할 것이다. data를 생성하여 그래프에 그리는 commit event와 help를 호출하는 right-click이다. OK를 선택하자.

3) 코드 메뉴에서 Generate>>All Code를 선택한다.



<Generate All Code dialog box>

4) CodeBuilder dialog의 몇몇 옵션들은 반드시 지정해 주어야한다. 먼저 프로그램이 시작될 때 보여주고 싶은 panel을 선택해야 한다. 이 프로그램에서는 하나의 panel만 선택할 수 있다.

※ 이 예제에서는 panel 변수명을 반드시 panelHandle로 해 주어야 한다.

5) dialog box의 하단 부분은 .uir 파일의 callback function을 보여준다. 프로그램 실행이 중지되었을 때의 기능들을 선택할 수 있다. Shutdown을 선택하여 체크표시가 옆에 오도록 한다.

6) OK버튼을 선택한다. 이것은 프로그램에 소스코드를 만들도록 CodeBuilder를 동기시킨다. 새로운 소스 창이 다음 코드와 같이 나타날 것이다.

7) sample4.c로 파일을 저장한다.

6. Using Function Panels and the Libraries

▶Setting up

1) File 메뉴의 Close로 다른 창들을 모두 닫는다.
2) Run 메뉴의 sample4.c를 열고 Break at First Statement의 check 항을 제거한다.
3) sample4.uir을 선택하여 최소화 시켜둔다.

<Analyzing the Source Code>

sample4는 완전한 프로그램이 아니기 때문에 코드를 추가하여 프로그램을 완성해 보자. 프로그램은 3가지의 기능으로 구성되어 있다.

<The main Function>

main 함수는 간단하게 구성되어 있으며 프로그램을 구성할 때 선택하여야 할 첫 번째 단계이다.

```
int main (int argc, char *argv[])
{
        if (InitCVIRTE (0, argv, 0) == 0)
                return -1;  /* out of memory */
        if ((panelHandle = LoadPanel (0, "sample4.uir", PANEL)) < 0)
                return -1;
        DisplayPanel (panelHandle);
        RunUserInterface ();
        return 0;
}
```

마지막 장에서 만든 user interface를 동작시키기 전에 먼저 하드디스크의 .uir 파일에서 메모리로 panel을 로드하여야 한다.

◆ LoadPanel 함수는 main 함수에서 이러한 동작을 실행한다.
◆ DisplayPanel 함수는 화면에 panel을 보여준다.
◆ RunUser Interface 함수는 user interface에서 C 프로그램으로 결과를 전달하기 위해 CVI를 활성화 시킨다.

<The AcquireData Function>

AcquireData 함수는 Acquire 버튼을 클릭할 때 마다 자동적으로 실행된다. 여기서 AcquireData 함수는 간단한 난수 데이터의 배열이다.

```
int CVICALLBACK AcquireData (int panel, int control, int event,
        void *callbackData, int eventData1, int eventData2)
{
    switch (event) {
        case EVENT_COMMIT:
            break;
        case EVENT_RIGHT_CLICK:
            break;
    }
    return 0;
}
```

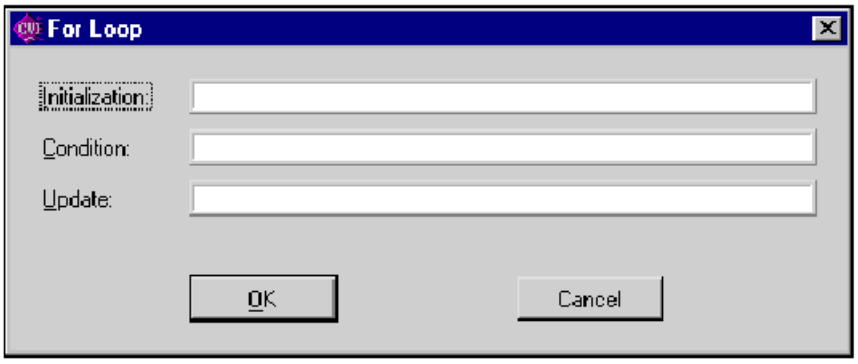
<The Shutdown Function>

Shutdown 함수는 Quit를 클릭하면 자동적으로 실행된다.

```
int CVICALLBACK Shutdown (int panel, int control, int event,
       void *callbackData, int eventData1, int eventData2)
{
    switch (event) {
       case EVENT_COMMIT:
          QuitUserInterface (0);
          break;
       case EVENT_RIGHT_CLICK:
          break;
    }
    return 0;
}
```

<Generating a Random Array of Data>

1) 소스 창의 맨위에 있는 datapoints와 i 변수를 다음과 같이 선언한다.
   int 1;
   double datapoints [100];
2) AcquireData 함수에 있는 EVENT_COMMIT에 입력커서를 위치시킨다.
3) CVI는 for 루프나 while 루프와 같은 공통 C 구조들을 만들어 준다. Edit 메뉴에서 Insert Construct를 선택하여 For loop를 선택하여 다음 그림이 나타나게 한다.

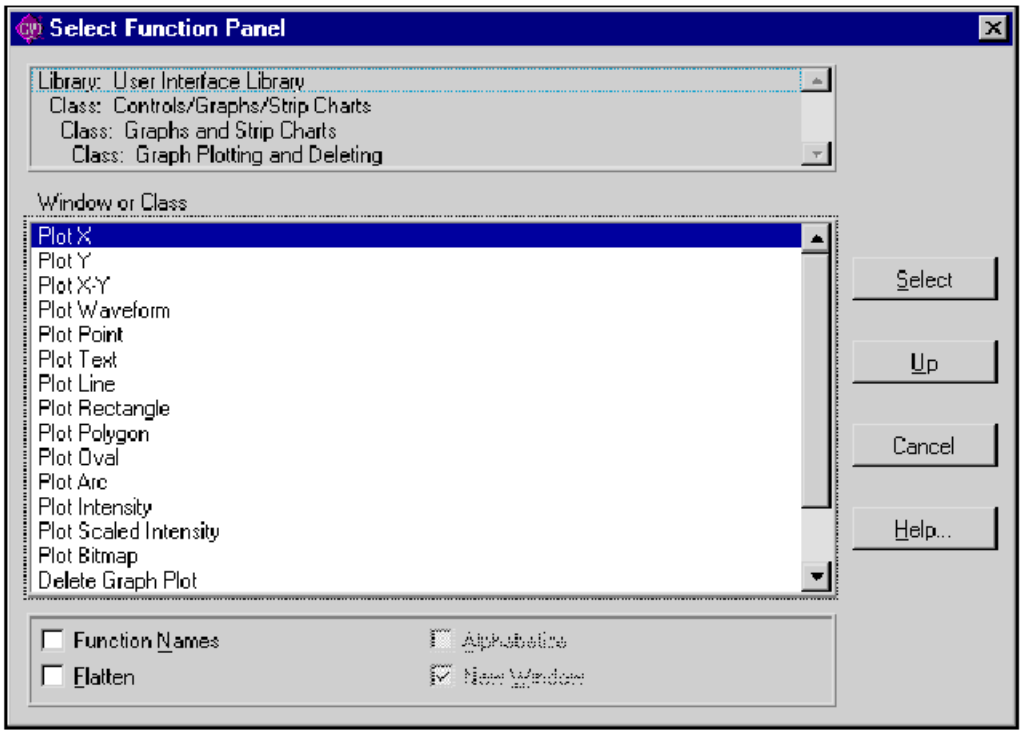


<그림 6-1 For loop dialog box>

4) For loop dialog에 다음을 입력한다.
   Initialization : i=0
   Condition : i<100
   Update : i++
   OK를 클릭한다.
5) 난수를 발생시키기 위해 다음의 코드를 for loop에 입력한다.
   datapoints[i] = rand()/32767.0;

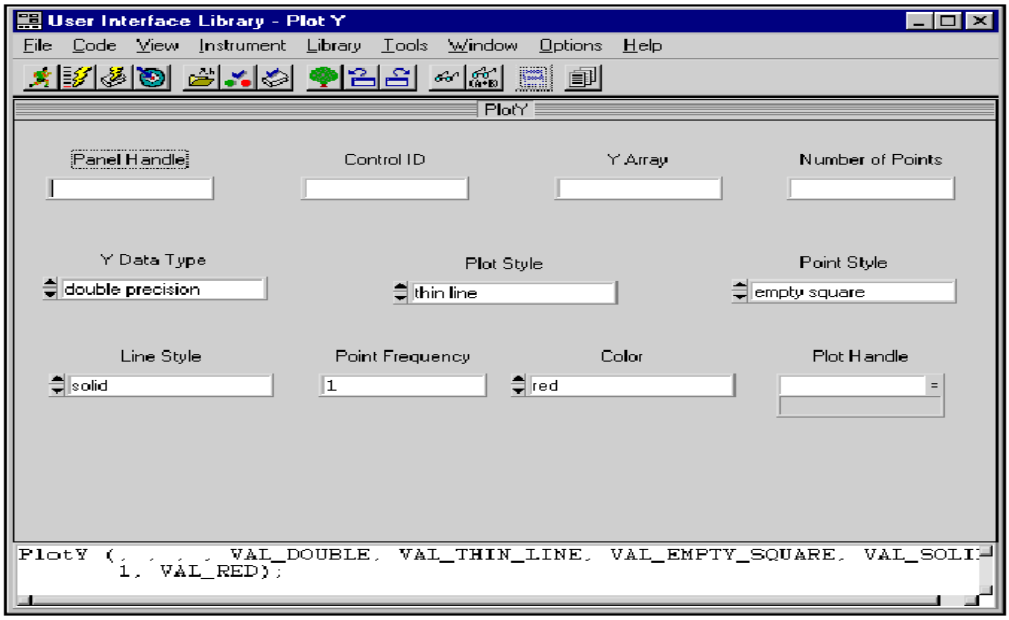
<Finding the PlotY Function>

1) AcquireData 함수의 datapoints[i] = rand()/32767.0의 바로 다음에 커서를 위치시킨다.
2) Library 메뉴의 User Interface를 선택한다.
3) Controls/Graphs/Strip Chart를 선택한다.
4) Graph and Strip Chart를 선택한다.
5) Graph Plotting and Deleting을 선택하여 Graph나 챠트에 관련된 모든 CVI 함수들을 나타나게 한다.



<그림 6-2 Graph Plotting and Deleting Function>

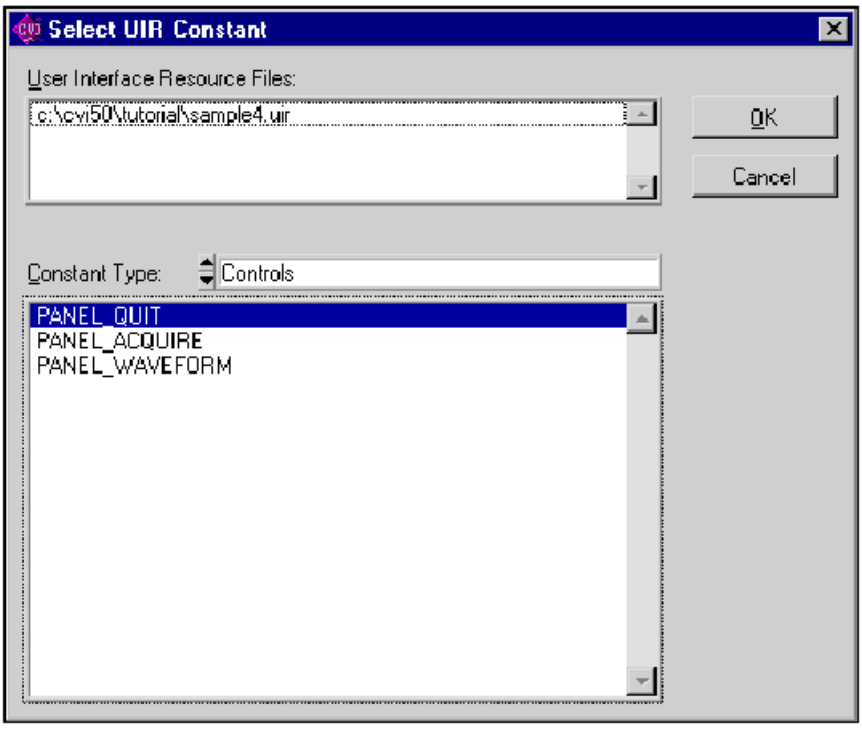
6) PlotY 함수를 선택한다.]



<그림 6-3 PlotY Function panel>

▶Building the PlotY Function Call Syntax

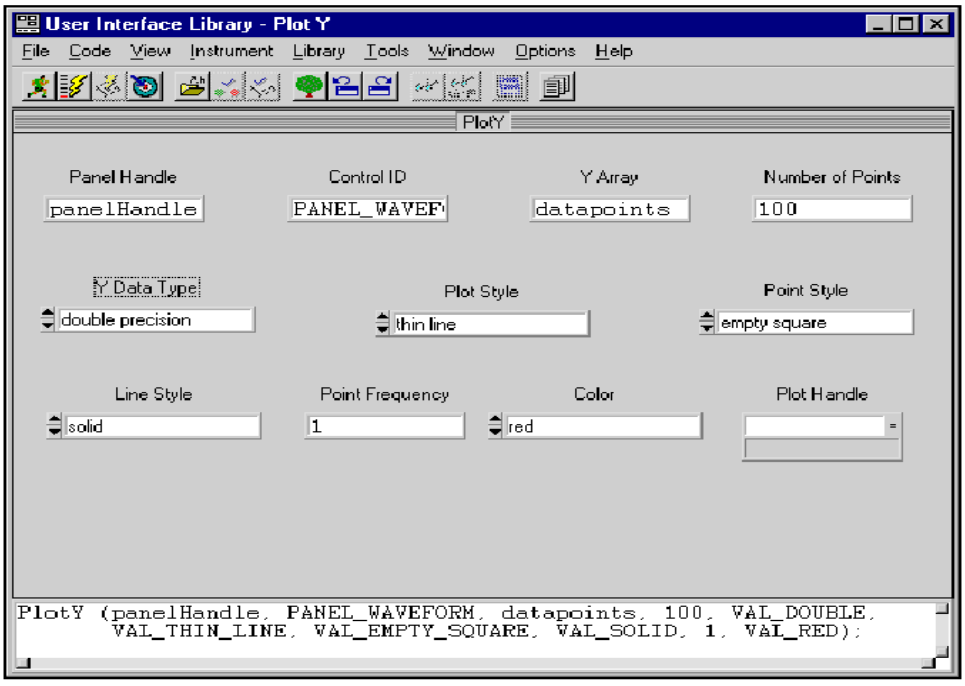
1) 커서를 panel handle control에 놓고 Code 메뉴의 Select Variable를 선택한다.
2) Control ID는 Graph control에서 지정된 상수명을 포함한다. 작업하고 있는 .uir 파일에서 모든 상수명을 보기위해 Code 메뉴의 Select UIR constant를 선택한다.



<그림 6-4 UIR constant dialog box>

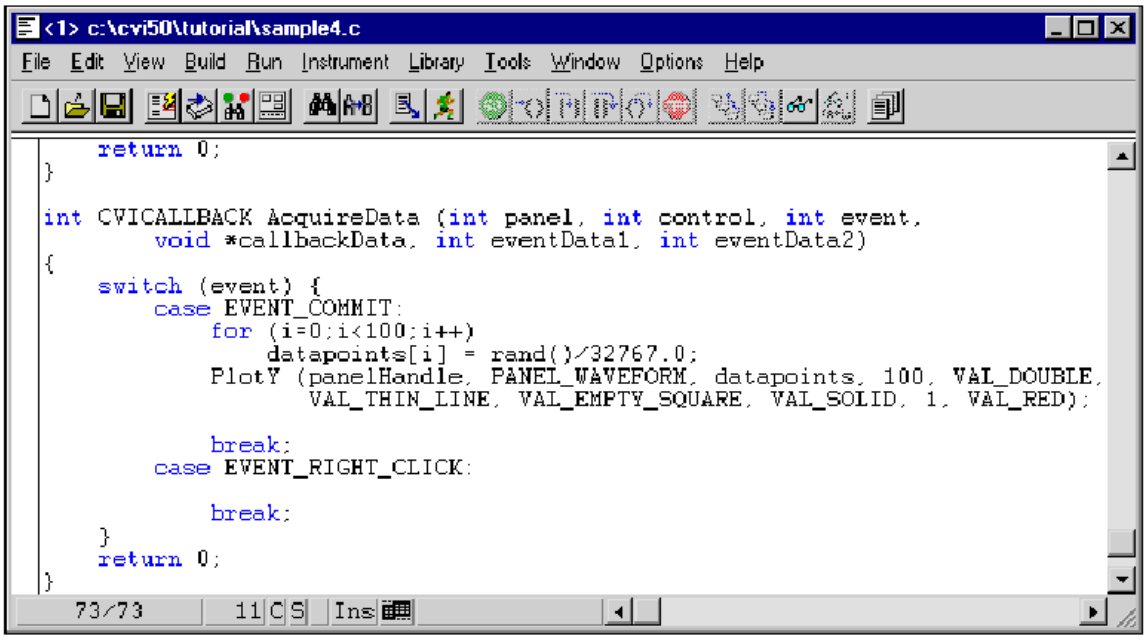
PANEL_WAVEFORM을 선택한다.

3) Y array에 datapoints를 입력한다. 이 이름은 그래프에 그려질 메모리 상의 배열을 가르킨다.
4) Number of point에 100을 입력한다.



<그림 6-5 완전한 Plot Y 함수 panel>

5) Code 메뉴에서 Set Target File을 선택하여 sample4.c를 지정한다.
6) Code 메뉴에서 Insert Function Call을 선택하여 소스코드에 삽입한다.
7) function panel을 닫는다.



<그림 6-6 완전한 sample4.c 소스코드>

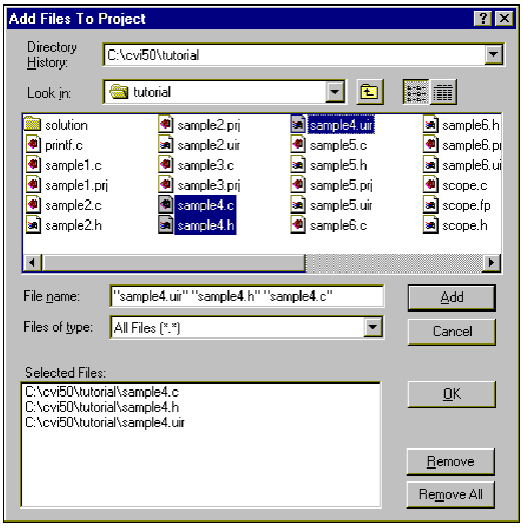
8) 파일을 save 시킨다.

▶Constructing the Project

프로그램의 만들기 위해서는 다음의 3파일이 project에 포함되어있어야 한다.

· sample4.uir : user interface 파일
· sample4.h : .uir파일을 저장할 때 자동적으로 생성되는 파일.
· sample4.c : 소스파일

1) project window을 제외한 모든 창을 닫는다.
2) Edit 메뉴에서 Add File to Preject를 선택한다. 모든 파일(*.*)타입을 선택한다.
3) sample4.c, sample4.h, sample4.uir을 선택한다.



<그림 6-7 Add file to project>

4) file을 save한다.

▶Running the Completed Project

프로그램을 실행시켜 보면 ansi_c.h가 빠져있다고 CVI가 메시지를 줄 것이다. YES를 선택하고 다시 save를 시킨다.

1) CVI는 sample4.c에서 코드를 컴파일하고 CVI library와 링크한다.
2) user interface가 나타나고 키보드나 마우스의 입력을 기다리게 된다.
3) Acquire 버튼을 눌렀을 때 CVI는 발생된 결과 정보를 AcquireData callback 함수로 넘겨준다.
4) Acquiredata 함수는 난수 데이터를 발생시키고 user interface에 그래프를 그린다.
5) Quit 버튼을 누르면 Shutdown 함수가 호출되어 프로그램이 정지된다.

7. Adding Analysis to Your Program

· User Interface Editor에서 uir에 control을 추가하는 방법을 재 고찰한다.
· callback function 사용법
· Analysis Library
· function panel을 사용하여 소스코드 작성 재 고찰
· user interface control로 수치 값 전달하기.
이번 장을 학습하기 위해서는 6장의 내용이 완전히 숙지되어 있어야 한다.

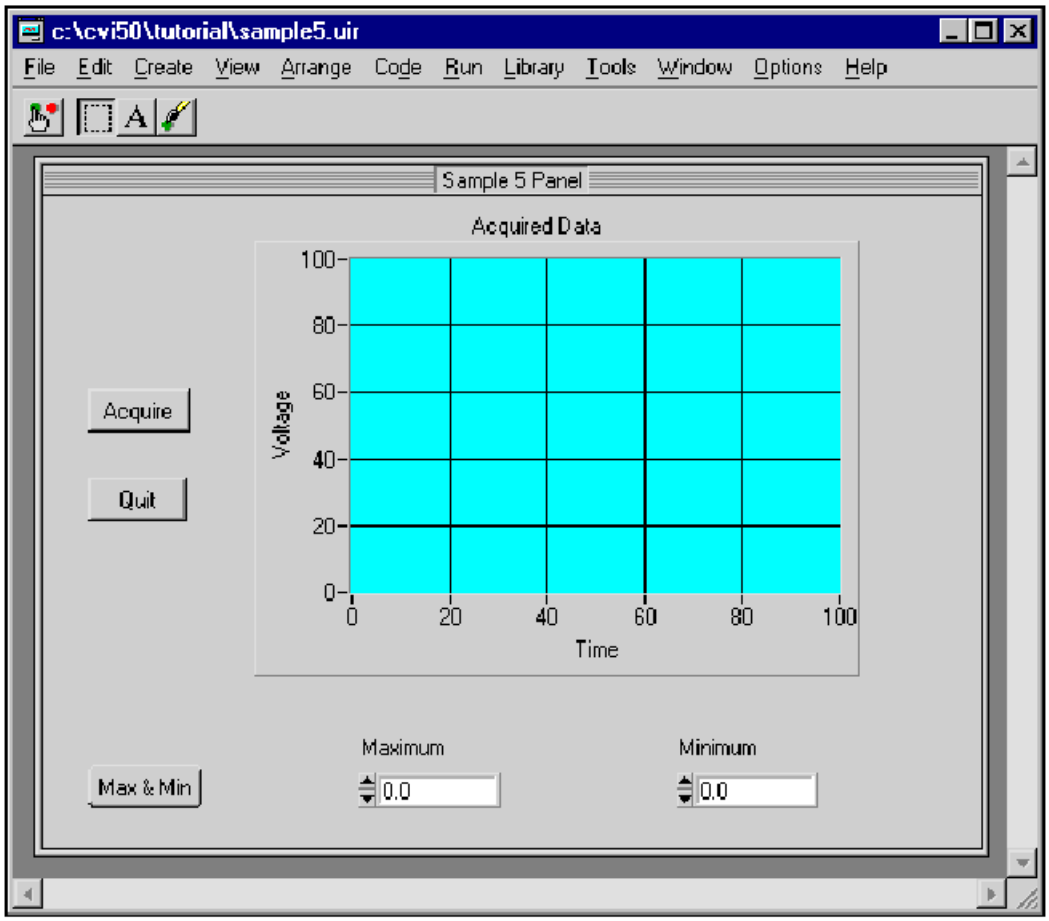
▶Setting up

1) project window을 띄워놓고 File 메뉴에서 Open을 선택하여 sample5.prj를 연다.
2) project를 실행하여 정상적으로 동작하는지를 확인한다.

<Modifying the User Interface>

먼저 5장에서 만든 user interface를 변형시켜 본다.

1) Project window에서 sample5.prj를 연다. 커서를 파일의 끝에 위치시킨다. 이 작업은 CodeBuilder에서 초기에 파일의 맨 밑에서 소스를 추가하도록 되어있으므로 임의로 파일 끝에 커서를 두지 않아도 될 경우도 있다. 여기에는 나중에 만들어질 새로운 callback function이 추가될 것이다.
2) sample5 소스를 닫지 않고, project wondow로 가서 sample5.uir을 클릭한다. 여기서 다음과 같이 uir을 변경시킬 것이다.



<그림 7-1 Sample user interface>

3) Creat 메뉴에서 Command Button을 선택하고 control palette에서 버튼을 선택한다.

4) Command 버튼을 더블클릭하여 dialog box가 나타나면 다음을 입력한다.

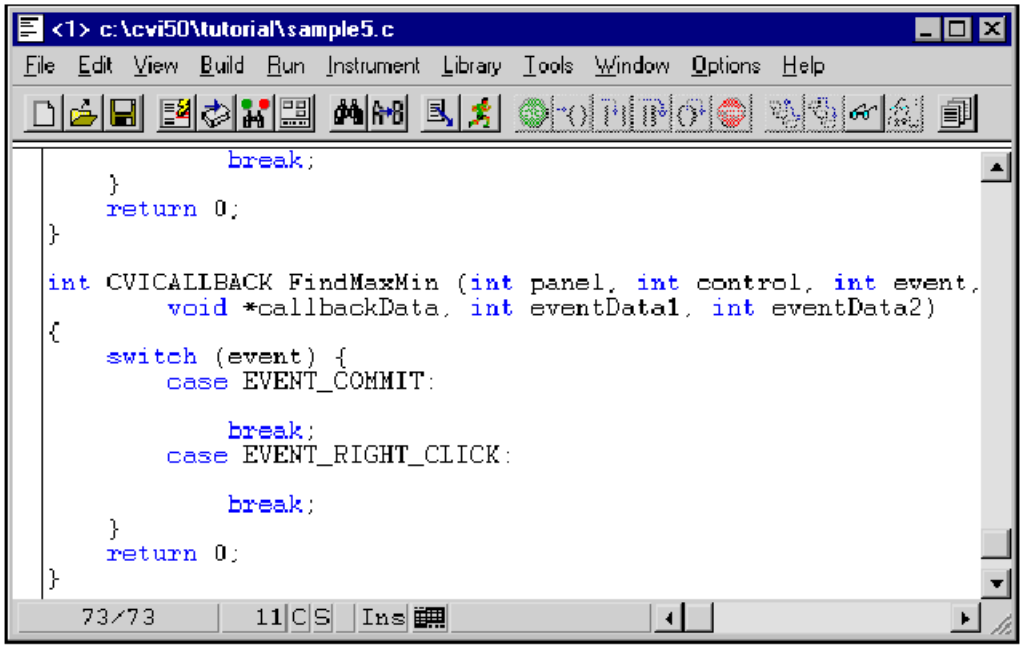
Constant Name : MAXMIN

Callback Function : FindMaxMin

Label : Max & Min

5) 특정한 control callback function을 위해 CodeBuilder를 사용하여 프로그램에 추가 할 수 있다.

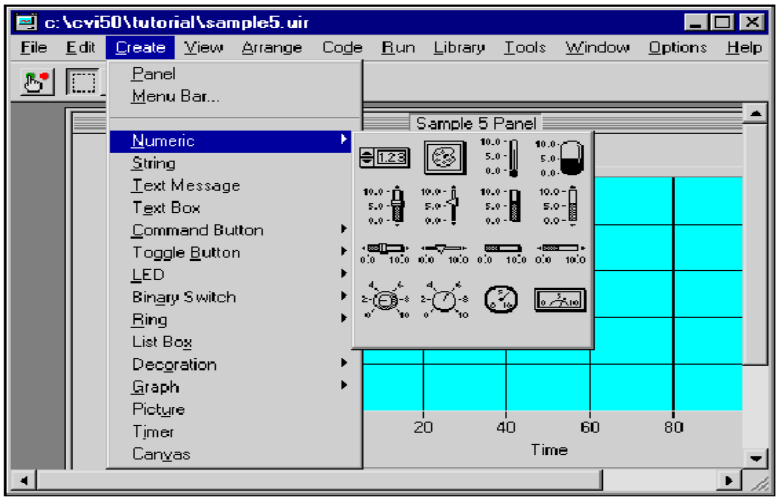
Max & Min 버튼을 Right-click하여 popup 메뉴에서 Generate Control Callback을 선택한다.



<그림 7-2 Generated Code>

위의 작업 다음으로 배열 값의 최대, 최소값을 계산하고 표시하기 위해 FindMaxMin callback function을 추가해야 한다.

Creat 메뉴에서 Numeric을 선택하고 그 중에서 Numeric control을 하나 선택한다.



<그림 7-3 Numeric control의 선택>

6) Numeric control을 더블 클릭하여 Edit dialog box가 나타나게 하여 다음을 입력한다.

Constant Name : MAX

Control Mode : Indicator

Label : Maximum

7) Create 메뉴에서 Numeric을 선택하여 왼쪽 상단의 control을 선택한다.

8) 더블 클릭하여 다음을 입력한다.

Constant Name : MIN

Control Mode : Indicator

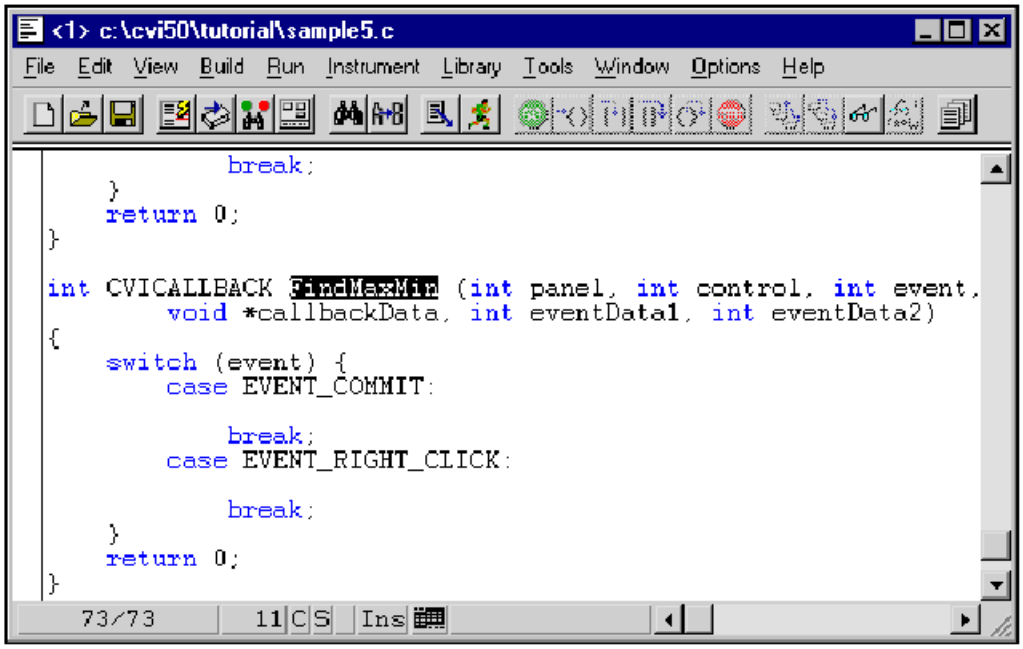
Label : Minimum

9) File 메뉴에서 save를 선택하여 .uir을 저장한다.

<Writing the Callback Function>

FindMaxMin 함수를 컴파일 하기 위해 다음을 실행한다.

1) CodeBuilder를 사용하여 신속하게 소스파일에 FindMaxMin callback function을 위치시킬 수 있다. CodeBuilder popup 메뉴를 다시 나타나게 하기위해 user interface editor의 Max & Min 버튼을 right-click한다. View Control Callback을 선택한다. CodeBuilder는 FindMaxMin callback function이 강조된 sample5.c 소스파일을 보여준다.



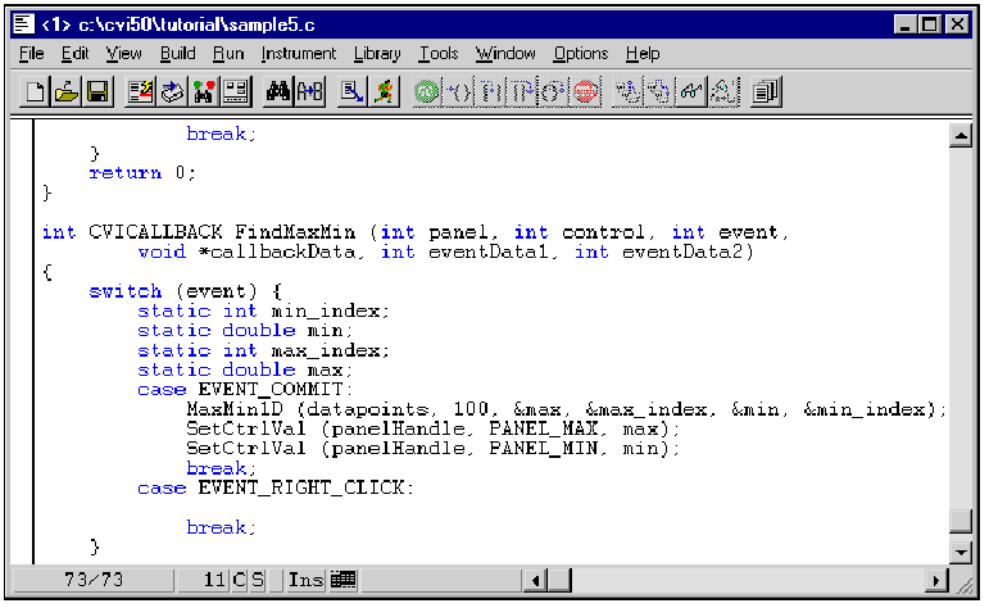
<그림 7-4 Using CodeBuilder to Find and Highlight the FindMaxMin Functionn>

2) EVENT_COMMIT 항 다음의 공백에 커서를 위치시킨다. 프로그램을 실행시키거나 Max & Min 버튼을 클릭할 때 if 항이 포함된 코드가 실행된다. user interface에서 배열의 최대 최소값을 찾아서 표시하기 위해 이 영역에 함수 호출을 입력해야 한다.

3) Library>Advanced Analysis>Array Operations>1D Operation>1D Maximum & Minimum

을 선택하여 Max & Min function panel을 표시한다.

4) MaxMin1D 함수는 배열의 maximum과 minimum을 찾는다. function panel의 control에 다음을 입력한다.
   Input Array : datapoints
   Number of Elements : 100
   Maximum Value : max
   Maximum Index : max_index
   Minimum Value : min
   Minimum Index : min_index
5) 소스코드로 MaxMin1D 함수를 삽입한 뒤, max, max_index, min, min_index를 선언해 주어야 한다. Maximum Value control에 클릭하여 code 메뉴에서 Declare Variable을 선택한다. check box에 Add declaration to current block를 체크한다. 이것은 FindMaxMin callback 함수의 max 변수를 선언하게 한다. OK 버튼을 클릭한다.
6) CVI는 자동적으로 max 변수 앞에 &를 삽입한다.
7) Declare Variable를 선택하여 나머지 변수들도 선언한다.
8) Code 메뉴에서 Insert Function Call을 선택하여 소스로 MaxMin1D function call을 삽입한다. MaxMin1D function panel을 닫고 FindMaxMin callback function안의 EVEVT_COMMIT 항의 아래에 MaxMin1D 함수가 삽입된 것을 확인한다.
9) Lirary>User Interface>Controls>Graphs/Strip Charts>General Function>Set Control Value를 선택하여 SetCtrlVal function panel을 나타나게 한다.
10)다음항목을 입력한다.
   Panel Handle : panelHandle
   Control ID : PANEL_MAX
   Value : max
11)Code 메뉴에서 Insert Function Call을 선택한다. 9) 단계를 반복하여 다음을 입력한다.
   Panel Handle : panelHandle
   Control ID : PANEL_MIN
   Value : min
12) Insert Function Call로 함수 삽입
13) 최종코드는 다음과 같을 것이다.



<그림 7-5 Completed Source Code for sample5.c>

<Running the Program>

사용자 callback function 작성을 성공적으로 마쳤다. 프로그램이 실행되는 동안, FindMaxMin 함수는 프로그램이 동작하거나 Max & Min 버튼을 클릭할 때 호출될 것이다. Max & Min 버튼을 클릭하면 다음 3가지 상황이 발생한다.

1) 먼저 버튼은 입력 focus를 얻는다(EVENT_GOT_FOCUS)
2) 왼쪽 마우스 클릭이 감지된다(EVENT_LEFT_CLICK)
3) 왼쪽 마우스 클릭을 놓는 것이 감지되고(EVENT_COMMIT) FindMaxMin 함수가 각 상황을 호출한다. COMMIT evevt가 감지되었을 때는 단지 Max & Min의 값을 구하고 표시하는 함수만 쓸 수 있다.

실행시 주의 할 점은 프로그램을 실행한 뒤에, Acquire 버튼을 눌러 난수를 발생시키고 그 다음 Max & Min 버튼을 눌러 다른 값들을 구한다.

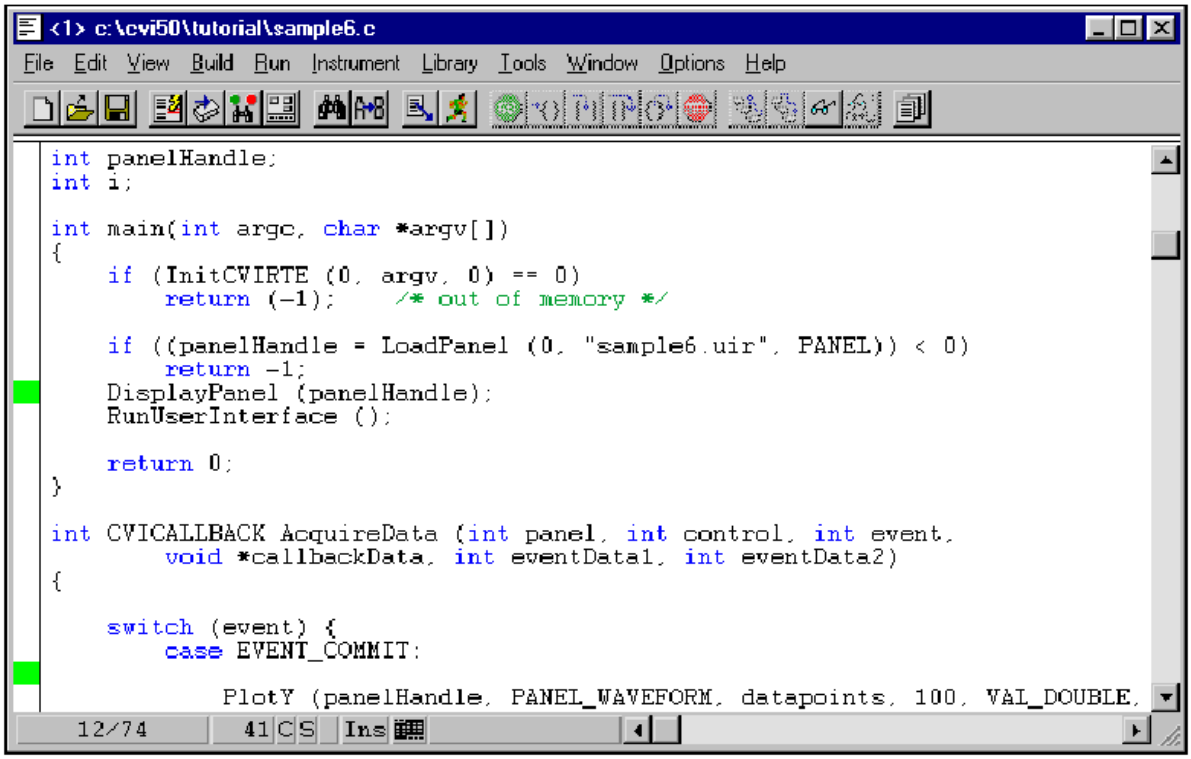
8. Using an Instrument Driver

sample6.prj를 선택하여 open한다.

▶Loading the Instrument Driver

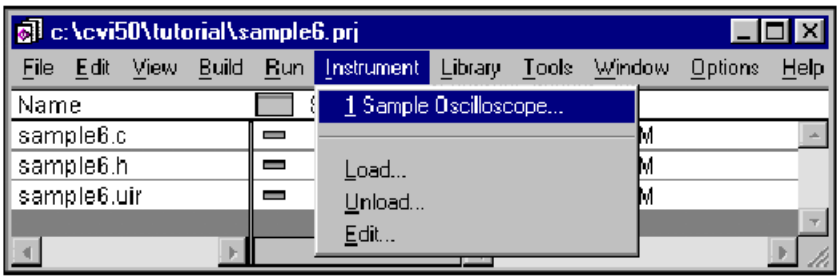
Instrument 메뉴를 사용하여 CVI에서 사용하는 파일을 Load 한다.

1) project window에서 Instrument 메뉴를 선택하여 scope.fp를 Load 한다.
2) sample6.c를 더블 클릭하여 source window에 코드가 나타나게 한다.
3) 커서를 main program의 중간부분의 DisplyPanel에 위치시킨다. <F2>를 사용하면 빨리 이동할 수 있다.



<8-1 Sample6.c>

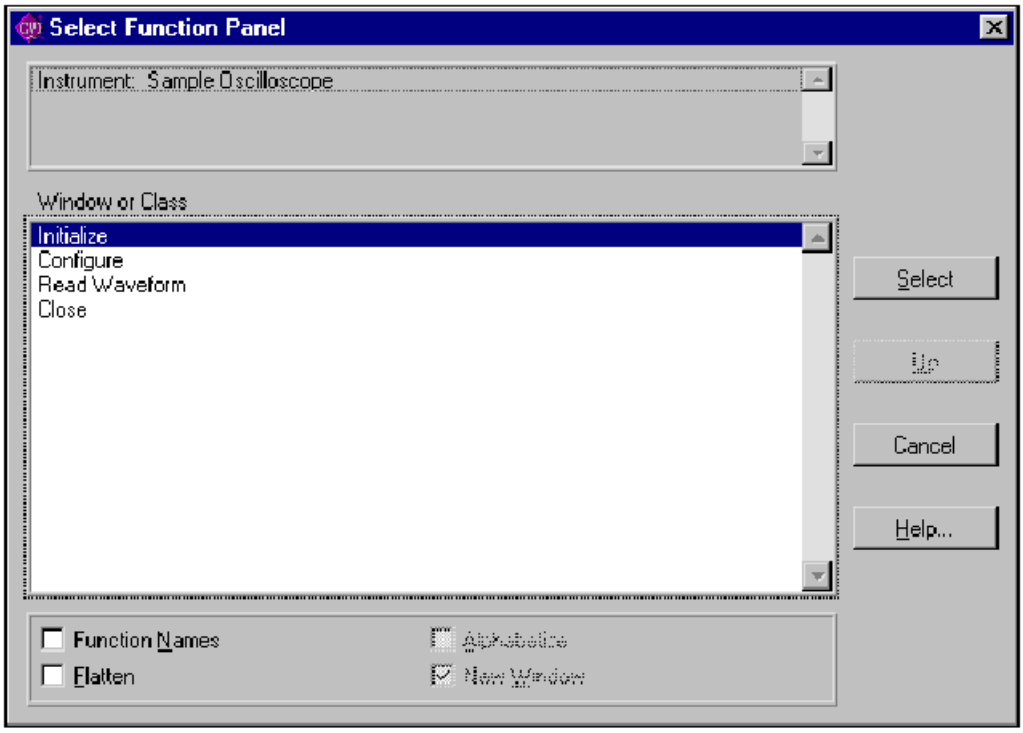
Scope Instrument driver가 load되었는지 확인하기 위해 Instrument 메뉴를 나타나게 한다. Sample Oscilloscope 항목이 다음 그림과 같이 나타난다.



<그림 8-2 Sample Oscilloscope on Instrument Menu>

▶Using the Instrument Driver

Instrument driver가 load되면 Instrument 메뉴에서 sample Oscilloscope를 선택하여 다음의 창이 나타나게 한다.



<그림 8-3 Sample Oscilloscope Functions>

이 모듈은 다음의 함수를 포함하고 있다.

· Initialize
· Configure
· Read Waveform
· Close

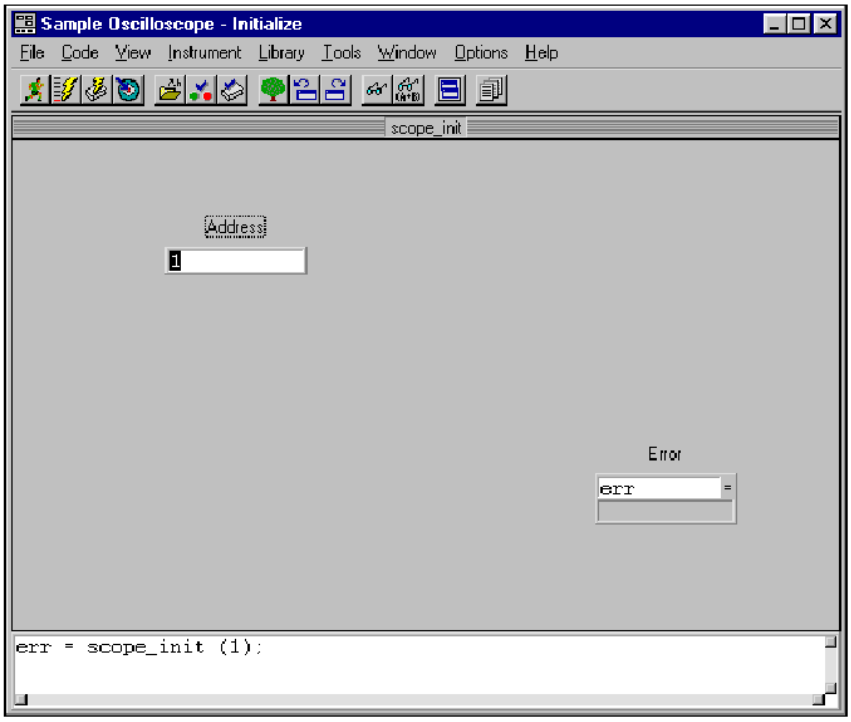
sample 프로그램에서 waveform을 취득하려면 이들 함수를 사용해야 한다. 이 dialog box를 닫고, 프로그램에 코드를 삽입하기 전에 어떻게 function panel들이 실행되는 지에 대해 좀더 읽어 본 뒤에 함수를 선택하도록 하자

<Interactive Function Panel Execution>

3장에서 function panel을 사용하여 개발하고자 하는 프로그램으로 코드를 생성하여 삽입하는 방법을 학습하였다. function panel의 다른 중요한 기능은 완벽한 프로그램의 작성없이도 panel 상호간의 함수를 실행할 수 있다는 것이다. 따라서 panel에서 parameter들을 변화시켜 가면서 만족스러운 결과를 얻을 때까지 함수들을 실행해 볼 수 있다.

<Initializing the Instrument>

각 장치 드라이버들은 소프트웨어와 장치들을 초기화 하기위한 함수를 사용한다. 모듈에서 다른 함수를 사용하기 전에 반드시 초기화 함수를 실행해야 한다. dialog box에서 Initialize를 선택하면 다음 화면이 나타난다.



<그림 8-4 Initialize Function Panel>

이 function panel은 장치의 GPIB 어드레스를 지정하기 위한 입력 control을 가지고 있다. Error control은 이 모듈의 동작과 관련된 error code를 표시하기 위해 사용된다.

1) Address control에 1을 입력한다.
2) Error control에 err을 입력한다.
3) Code 메뉴에서 Declare Variable을 선택하여 err 변수를 선언한다. Execute declaration and Add declaration to the top of target file항에 체크 표시를 확인한다.
4) Code 메뉴에서 Run Function Panel을 선택한다. 변화된 파일을 저장한다.

에러가 발생하지 않았다면 Error control의 값은 0가된다. 만약 0가 아니라면 Error control에 관련한 도움말을 참조한다.

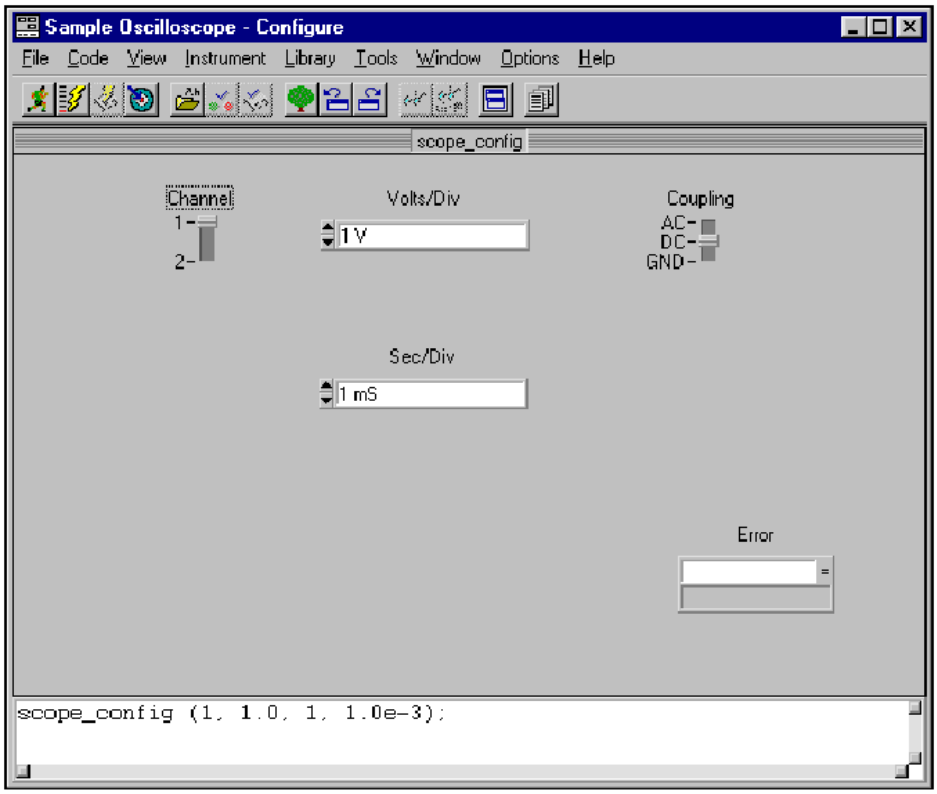
1) Code 메뉴에서 Insert Function Call을 선택하여 source window로 코드를 복사한다.
2) 삽입코드에 대한 target 파일을 지정해야 하는데, 코드를 삽입할 때 dialog box가 나타나면 sample6.c를 선택해준다.
3) File 메뉴에서 close 선택

장치 드라이버를 초기화하는 함수 호출이 source window의 DisplayPanel function call 위에 나타날 것이다.

[err = scope_init (1) ;]

<Configuring the Instrument>

장치 초기화가 끝나면 waveform을 읽어서 프로그램의 배열로 전달하도록 장치를 형성할 수 있다. 오실로스코프 예제에서는 오실로스코프의 수평, 수직 파라메터를 configure function을 사용하여 설정할 수 있다. Instrument 메뉴에서 Sample Oscilloscope를 선택한다. dialog box에서 Configure를 선택하면 다음 화면이 나타난다.



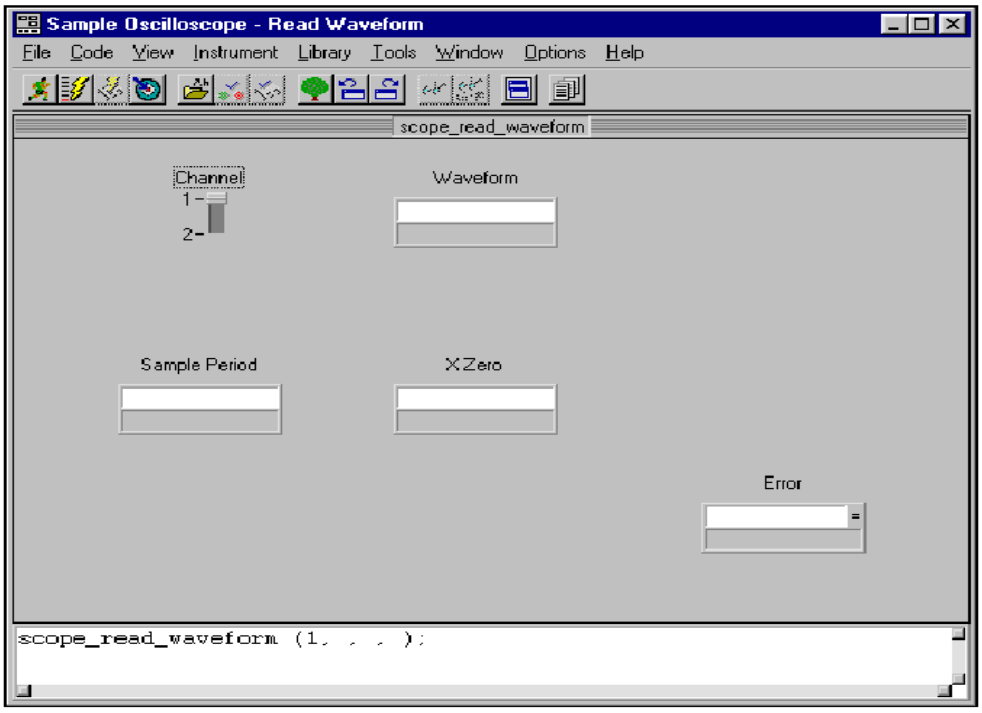
<그림 8-5 Configure Function Panel>

1) Error control에 err을 입력한다.
2) Code 메뉴에서 Run Function Panel을 선택하여 function panel을 실행시킨다.
   Error control이 0이 아니면 에러가 발생한 것이나 도움말을 참조하여 문제점을 해결한다.
   panel에 real instrument가 붙어 있다면, Run Function Panel을 선택했을 때 발생된 장치의 구조를 볼 수도 있다.
3) Code 메뉴에서 Insert Function Call을 선택하여 source window로 code를 복사한다. function panel을 닫는다.
4) 입력 커서를 AcquireData 함수의 PlotY 함수앞의 공백이 위치시켜둔다.

<Reading Data with an Instrument Driver>

Instrument driver의 가장 중요한 기능은 장치로부터 data를 읽고 프로그램에서 바로 사용가능한 raw data format으로 변환하는 것이다. 예를들어 디지털 오실로스코프는 waveform을 comma-separated ASCII 숫자열이나 2진 정수로 바꾼다.

Instrument 메뉴에서 Sample Oscilloscope의 Read Waveform을 선택한다.



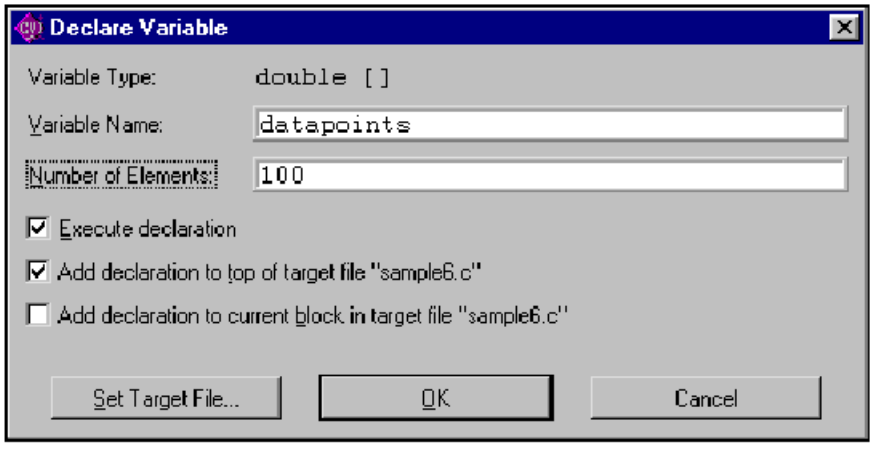
<그림 8-6 Read Waveform Function Panel>

읽기를 원하는 채널을 세팅한다. 채널 1은 정현파이고 채널 2는 난수 data이다.

<Declaring Arrays from Function Panels>

Read Waveform 함수는 파형 data를 배열에 위치시킨다. 함수를 실행하기 전에 파형에 대한 배열을 선언해야 한다. function panel로부터 변수를 선언할 수 있다.

1) Waveform control에서 datapoints를 입력한다.
2) datapoints 변수를 사용하기 위해 먼저 메모리에 이것을 선언해야한다. Code 메뉴에서 Declare Variable을 선택한다. Variable Name 입력난에 자동으로 datapoints가 입력된다.
3) Number of Elements에 100을 입력한다.
4) Add declaration to the top of target file sample6.c 옵션에 체크표시를 확인한다.
5) <Enter>로 datapoints array를 선언한다.



<그림 8-7 Declare Variable Dialog Box>

<Reading the Waveform>

1) sample period에 커서를 위치시키고 코드메뉴에서 Declare Variable을 선택한다.
2) delta_t를 입력한다.
3) X Zero control에 커서를 위치시키고 Declare Variable을 선택하여 x_zero를 입력한다.
4) err control에 err을 입력한다.
5) function panel을 실행시키기 위해 code 메뉴의 Run Function Panel을 선택한다. Error control이 0이 아니면 문제를 수정하여 0이 되도록 한다. 함수가 실행되면 Waveform control box 아래에 data가 표시된다.
6) source window로 코드를 삽입하기위해 Code 메뉴의 Insert Function Call을 선택한다. background의 source window를 클릭하여 plotY 함수앞에 코드가 생성되었는지를 확인한다.

<Closing the Instrument>

1) source window에서 커서를 shutdowm function의 QuitUserInterface(0);에 위치시킨다.
2) Instrument 메뉴에서 sample oscilloscope를 선택하여 close를 선택한다.
3) Error control에 error을 입력한다.
4) instrument driver를 닫기위해 Code 메뉴에서 run function panel을 선택한다.
5) source window로 코드를 삽입하기 위해 Code 메뉴의 Insert Function Call을 선택한다.
6) source window를 활성화 시켜 다음 코드를 확인한다.

Your program source code should match the following text.

```
static double x_zero;
static double delta_t;
static double datapoints[100];
static int err;

#include <cvirte.h>
#include <analysis.h>
#include <ansi_c.h>
#include <userint.h>
#include "sample6.h"
#include "scope.h"

int panelHandle;
int i;
int main(int argc, char *argv[])
{
   if (InitCVIRTE (0, argv, 0) == 0)
      return (-1);    /* out of memory */

   if ((panelHandle = LoadPanel (0, "sample6.uir", PANEL)) < 0)
      return -1;
   err = scope_init (1);
   err = scope_config (1, 1.0, 1, 1.0e-3);
   DisplayPanel (panelHandle);
   RunUserInterface ();

   return 0;
}

int CVICALLBACK AcquireData (int panel, int control, int event,
      void *callbackData, int eventData1, int eventData2)
{

   switch (event) {
      case EVENT_COMMIT:
         err = scope_read_waveform (1, datapoints, &delta_t,
            &x_zero);
```

```
            PlotY (panelHandle, PANEL_WAVEFORM, datapoints, 100,
               VAL_DOUBLE, VAL_THIN_LINE, VAL_EMPTY_SQUARE,
               VAL_SOLID, 1, VAL_RED);
            break;
         case EVENT_RIGHT_CLICK:
            break;
      }
      return 0;
}
int CVICALLBACK FindMaxMin (int panel, int control, int event,
         void *callbackData, int eventData1, int eventData2)
{
      double max, min;
      int max_index, min_index;
      switch (event) {
         case EVENT_COMMIT:
            MaxMin1D (datapoints, 100, &max, &max_index, &min,
               &min_index);
            SetCtrlVal (panelHandle, PANEL_MAX, max);
            SetCtrlVal (panelHandle, PANEL_MIN, min);
            break;
         case EVENT_RIGHT_CLICK:
            break;
      }
      return 0;
}
int CVICALLBACK Shutdown (int panel, int control, int event,
         void *callbackData, int eventData1, int eventData2)
{
      switch (event) {
         case EVENT_COMMIT:
            err = scope_close ();
            QuitUserInterface (0);
            break;
         case EVENT_RIGHT_CLICK:
            break;
      }
      return 0;
}
```

프로그램을 실행하기 전에 요구되는 마지막 단계는 scope header file을 추가하는 것이다. scope instrument driver로부터 함수를 호출하기 위해서는 source file의 맨 윗부분에 다음을 추가하여야 한다.

#include "scope.h"

Run 메뉴에서 Run Project를 실행한다.

<Adding the Instrument to Your Project>

Instrument 메뉴로 scope drive를 로드하면 CVI의 instrument driver에 function panel을 직접 추가할 수 있다. project의 file list에 scope driver를 추가함으로써 scope driver function panel은 추후에 project를 열 때 Instrument 메뉴에 자동적으로 추가 될 것이다.

1) project window를 제외한 모든 창을 닫는다.
2) Edit 메뉴에서 Add File to Project를 선택하고 instrument(*.ft)를 지정한다.
3) dialog box에서 scope.fp를 선택한다.